

Práctica 2.1 – Instalación y configuración de servidor web Nginx

Instalación servidor web Nginx

Para instalar el servidor nginx en nuestra Debian, primero actualizamos los repositorios y después instalamos el paquete correspondiente:

```
sudo apt update
```

```
sudo apt install nginx
```

Comprobamos que nginx se ha instalado y que está funcionando correctamente:

```
systemctl status nginx
```

Esta práctica se ha hecho con Nginx 1.18.0

Creación de las carpetas del sitio web

Igual que ocurre en Apache, todos los archivos que formarán parte de un sitio web que servirá nginx se organizarán en carpetas. Estas carpetas, típicamente están dentro de `/var/www`.

Así pues, vamos a crear la carpeta de nuestro sitio web o dominio:

```
sudo mkdir -p /var/www/nombre_web/html
```

Donde el nombre de dominio puede ser la palabra que queráis, sin espacios.

Ahí, dentro de esa carpeta html, debéis clonar el siguiente repositorio:

<https://github.com/cloudacademy/static-website-example>

Además, haremos que el propietario de esta carpeta y todo lo que haya dentro sea el usuario `www-data`, típicamente el usuario del servicio web.

```
sudo chown -R www-data:www-data /var/www/nombre_web/html
```

Y le daremos los permisos adecuados para que no nos de un error de acceso no autorizado al entrar en el sitio web:

```
sudo chmod -R 755 /var/www/nombre_web
```

Para comprobar que el servidor está funcionando y sirviendo páginas correctamente, podéis acceder desde vuestro cliente a:

```
http://IP-maq-virtual
```

Y os deberá aparecer algo así:



Lo que demuestra que todo es correcto hasta ahora.

Configuración de servidor web NGINX

En Nginx hay dos rutas importantes. La primera de ellas es `sites-available`, que contiene los archivos de configuración de los hosts virtuales o bloques disponibles en el servidor. Es decir, cada uno de los sitios webs que alberga el servidor. La otra es `sites-enabled`, que contiene los archivos de configuración de los sitios habilitados, es decir, los que funcionan en ese momento.

Dentro de `sites-available` hay un archivo de configuración por defecto (`default`), que es la página que se muestra si accedemos al servidor sin indicar ningún sitio web o cuando el sitio web no es encontrado en el servidor (debido a una mala configuración, por ejemplo). Esta es la página que nos ha aparecido en el apartado anterior.

Para que Nginx presente el contenido de nuestra web, es necesario crear un bloque de servidor con las directivas correctas. En vez de modificar el

archivo de configuración predeterminado directamente, crearemos uno nuevo en `/etc/nginx/sites-available/nombre_web`:

```
sudo nano /etc/nginx/sites-available/vuestro_dominio
```

Y el contenido de ese archivo de configuración:

```
server {  
    listen 80;  
    listen [::]:80;  
    root /ruta/absoluta/archivo/index;  
    index index.html index.htm index.nginx-debian.html;  
    server_name nombre_web;  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

Aquí la directiva `root` debe ir seguida de la ruta absoluta absoluta dónde se encuentre el archivo `index.html` de nuestra página web, que se encuentra entre todos los que habéis descomprimido.

Aquí tenéis un ejemplo de un sitio webs con su ruta (directorios que hay) antes del archivo `index.html`:



Info

Ruta → /var/www/ejemplo2/html/2016_soft_landing

Y crearemos un archivo simbólico entre este archivo y el de sitios que están habilitados, para que se dé de alta automáticamente.

```
sudo ln -s /etc/nginx/sites-available/nombre_web /etc/nginx/sites-enabled/
```

Y reiniciamos el servidor para aplicar la configuración:

```
sudo systemctl restart nginx
```

Comprobaciones

Comprobación del correcto funcionamiento

Como aún no poseemos un servidor DNS que traduzca los nombres a IPs, debemos hacerlo de forma manual. Vamos a editar el archivo `/etc/hosts` **de nuestra máquina anfitriona** para que asocie la IP de la máquina virtual, a nuestro `server_name`.

Este archivo, en Linux, está en: `/etc/hosts`

Y en Windows: `C:\Windows\System32\drivers\etc\hosts`

Y deberemos añadirle la línea:

`192.168.X.X nombre_web`

donde debéis sustituir la IP por la que tenga vuestra máquina virtual.

Comprobar registros del servidor

Comprobad que las peticiones se están registrando correctamente en los archivos de logs, tanto las correctas como las erróneas:

- `/var/log/nginx/access.log`: cada solicitud a su servidor web se registra en este archivo de registro, a menos que Nginx esté configurado para hacer algo diferente.
- `/var/log/nginx/error.log`: cualquier error de Nginx se asentará en este registro.

Info

Si no os aparece nada en los logs, podría pasar que el navegador ha cacheado la página web y que, por tanto, ya no está obteniendo la página del navegador sino de la propia memoria. Para solucionar esto, podéis acceder con el *modo privado* del navegador y ya os debería registrar esa actividad en los logs.

FTP

Si queremos tener varios dominios o sitios web en el mismo servidor nginx (es decir, que tendrán la misma IP) debemos repetir todo el proceso anterior con el nuevo nombre de dominio que queramos configurar.

¿Cómo transferir archivos desde nuestra máquina local/anfitrión a nuestra máquina virtual Debian/servidor remoto?

A día de hoy el proceso más sencillo y seguro es a través de Github como hemos visto antes.

El **FTP** es un protocolo de transferencia de archivos entre sistemas conectados a una red TCP. Como su nombre indica, se trata de un protocolo que permite transferir archivos directamente de un dispositivo a otro. Actualmente, es un protocolo que poco a poco va abandonándose, pero ha estado vigente más de 50 años.

El protocolo FTP tal cual es un protocolo inseguro, ya que su información no viaja cifrada. Sin embargo, en 2001 esto se solucionó con el protocolo **SFTP**, que le añade una capa SSH para hacerlo más seguro y privado.

SFTP no es más que el mismo protocolo FTP pero implementado por un canal seguro. Son las siglas de SSH File Transfer Protocol y consiste en una extensión de Secure Shell Protocol (SSH) creada para poder hacer transmisiones de archivos.

La seguridad que nos aporta **SFTP** es importante para la transferencia de archivos porque, si no disponemos de ella, los archivos viajarán tal cual, por la red, sin ningún tipo de encriptación. Así pues, usando FTP tradicional, si algún agente consigue escuchar las transferencias, podría ocurrir que la información quedase al descubierto. Esto sería especialmente importante si los archivos que subimos contienen información confidencial o datos personales.

Dado que usar **SFTP** aporta mayor seguridad a las transmisiones, es recomendable utilizarlo, más aún sabiendo que realmente no hay mucha dificultad en establecer las conexiones por el protocolo seguro.

Configurar servidor SFTP en Debian

En primer lugar, lo instalaremos desde los repositorios:

```
sudo apt-get update
```

```
sudo apt-get install vsftpd
```

Ahora vamos a crear una carpeta en nuestro *home* en Debian:

```
mkdir /home/nombre_usuario/ftp
```

En la configuración de *vsftpd* indicaremos que este será el directorio al cual *vsftpd* se cambia después de conectarse el usuario.

Ahora vamos a crear los certificados de seguridad necesarios para aportar la capa de cifrado a nuestra conexión (algo parecido a HTTPS)

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/vsftpd.pem -  
out /etc/ssl/private/vsftpd.pem
```

Y una vez realizados estos pasos, procedemos a realizar la configuración de *vsftpd* propiamente dicha. Se trata, con el editor de texto que más os guste, de editar el archivo de configuración de este servicio, por ejemplo, con *nano*:

```
sudo nano /etc/vsftpd.conf
```

En primer lugar, buscaremos las siguientes líneas del archivo y las eliminaremos por completo:

```
rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
```

```
rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
```

```
ssl_enable=NO
```

Tras ello, añadiremos estas líneas en su lugar

```
rsa_cert_file=/etc/ssl/private/vsftpd.pem
```

```
rsa_private_key_file=/etc/ssl/private/vsftpd.pem
```

```
ssl_enable=YES
```

```
allow_anon_ssl=NO
```

```
force_local_data_ssl=YES
```

```
force_local_logins_ssl=YES
```

```
ssl_tlsv1=YES
```

```
ssl_sslv2=NO
```

```
ssl_sslv3=NO
```

```
require_ssl_reuse=NO
```

```
ssl_ciphers=HIGH
```

```
local_root=/home/nombre_usuario/ftp
```

Y, tras guardar los cambios, reiniciamos el servicio para que coja la nueva configuración:

```
sudo systemctl restart --now vsftpd
```

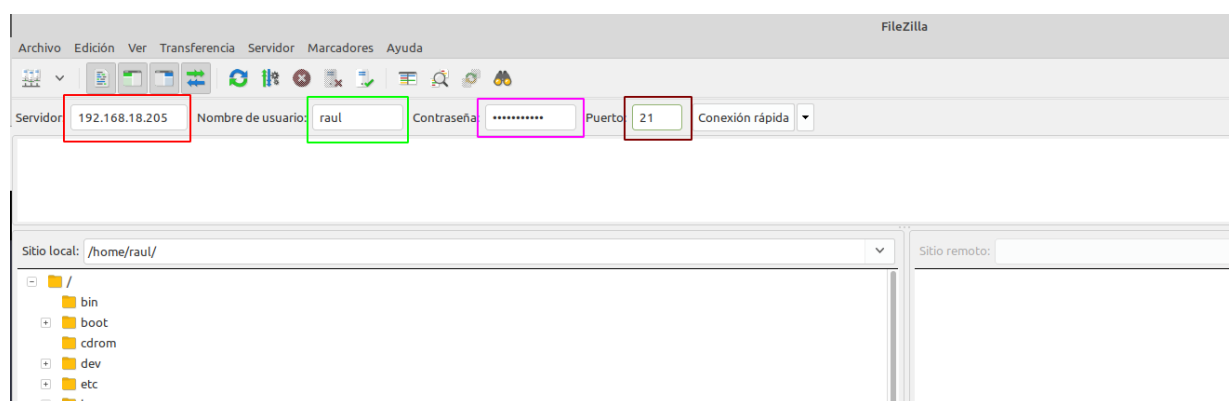
Tarea

Configura un nuevo dominio (nombre web) para el .zip con el nuevo sitio web que os proporcionado. **En este caso debéis transferir los archivos a vuestra Debian mediante SFTP.**

Tras acabar esta configuración, ya podremos acceder a nuestro servidor mediante un cliente FTP adecuado, como por ejemplo [Filezilla](#) de dos formas, a saber:

- Mediante el puerto por defecto del protocolo inseguro FTP, el 21, pero utilizando certificados que cifran el intercambio de datos convirtiéndolo así en seguro
- Haciendo uso del protocolo *SFTP*, dedicado al intercambio de datos mediante una conexión similar a SSH, utilizando de hecho el puerto 22.

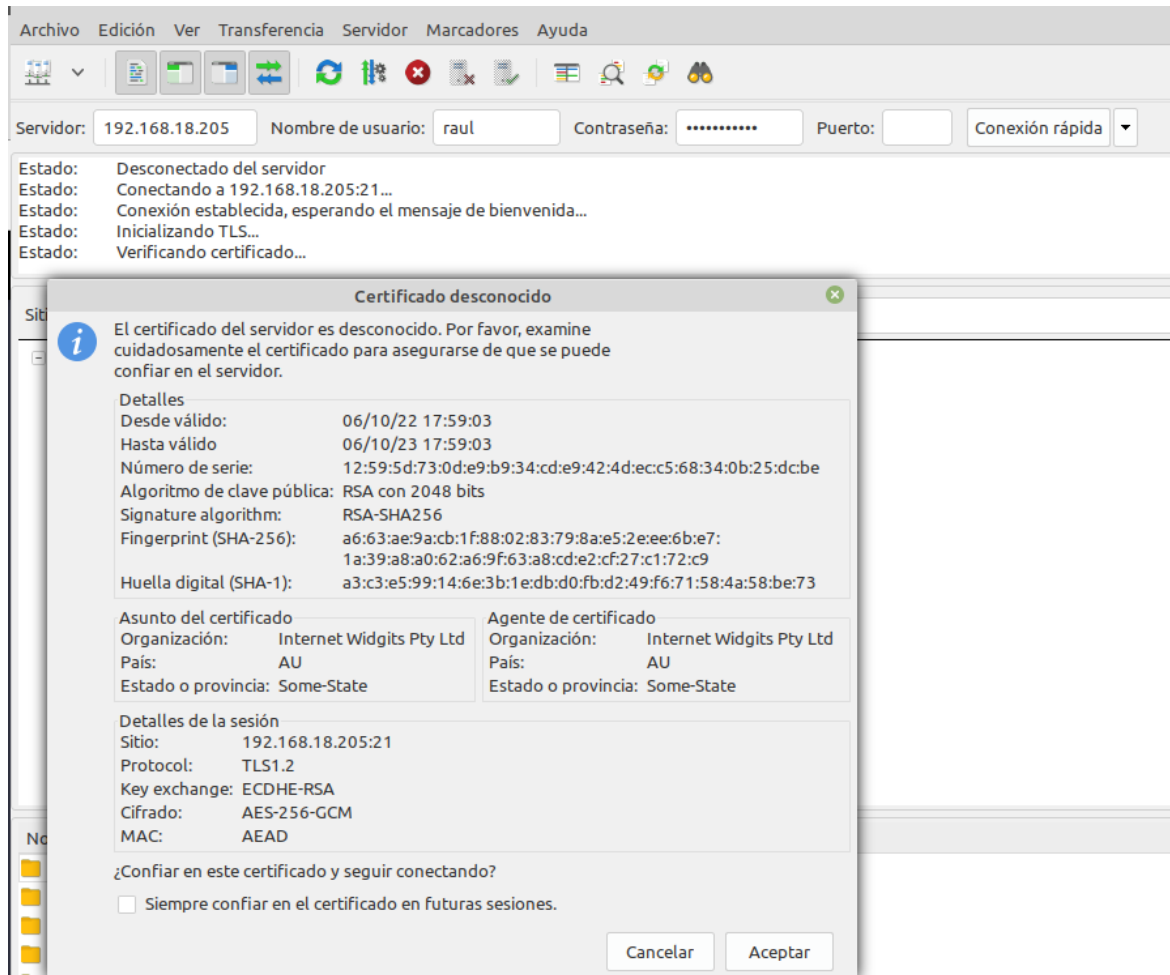
Tras descargar **el cliente FTP** en nuestro ordenador, introducimos los datos necesarios para conectarnos a nuestro servidor FTP en Debian:



- La IP de Debian (recuadro rojo)
- El nombre de usuario de Debian (recuadro verde)
- La contraseña de ese usuario (recuadro fucsia)

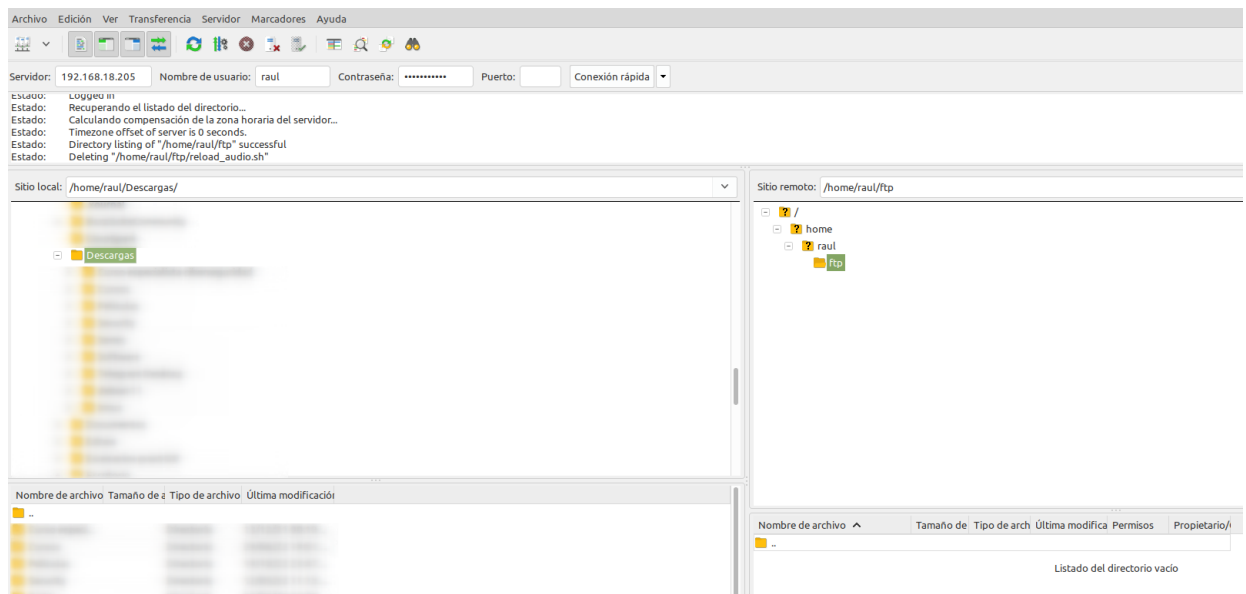
- El puerto de conexión, que será el 21 para conectarnos utilizando los certificados generados previamente (recuadro marrón)

Tras darle al botón de *Conexión rápida*, nos saltará un aviso a propósito del certificado, le damos a aceptar puesto que no entraña peligro ya que lo hemos generado nosotros mismos:

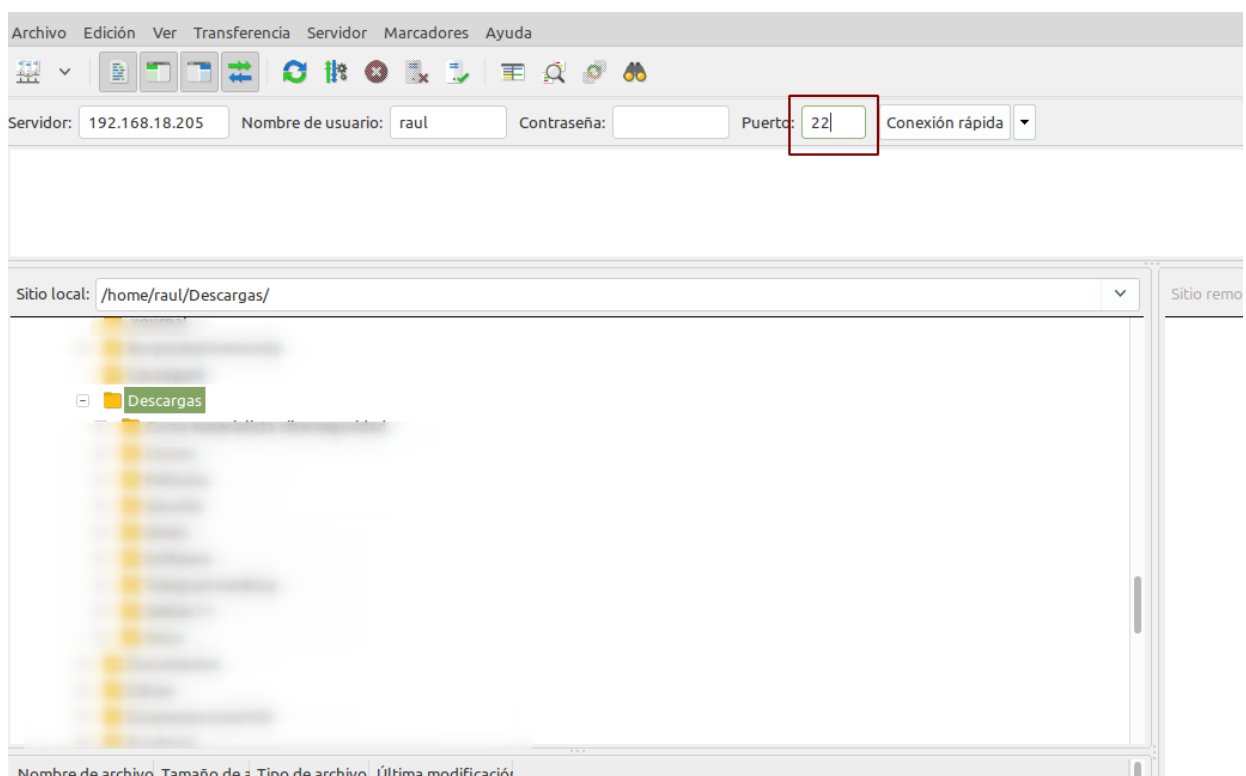


Nos conectaremos directamente a la carpeta que le habíamos indicado en el archivo de configuración `/home/raul/ftp`

Una vez conectados, buscamos la carpeta de nuestro ordenador donde hemos descargado el `.zip` (en la parte izquierda de la pantalla) y en la parte derecha de la pantalla, buscamos la carpeta donde queremos subirla. Con un doble click o utilizando *botón derecho > subir*, la subimos al servidor.



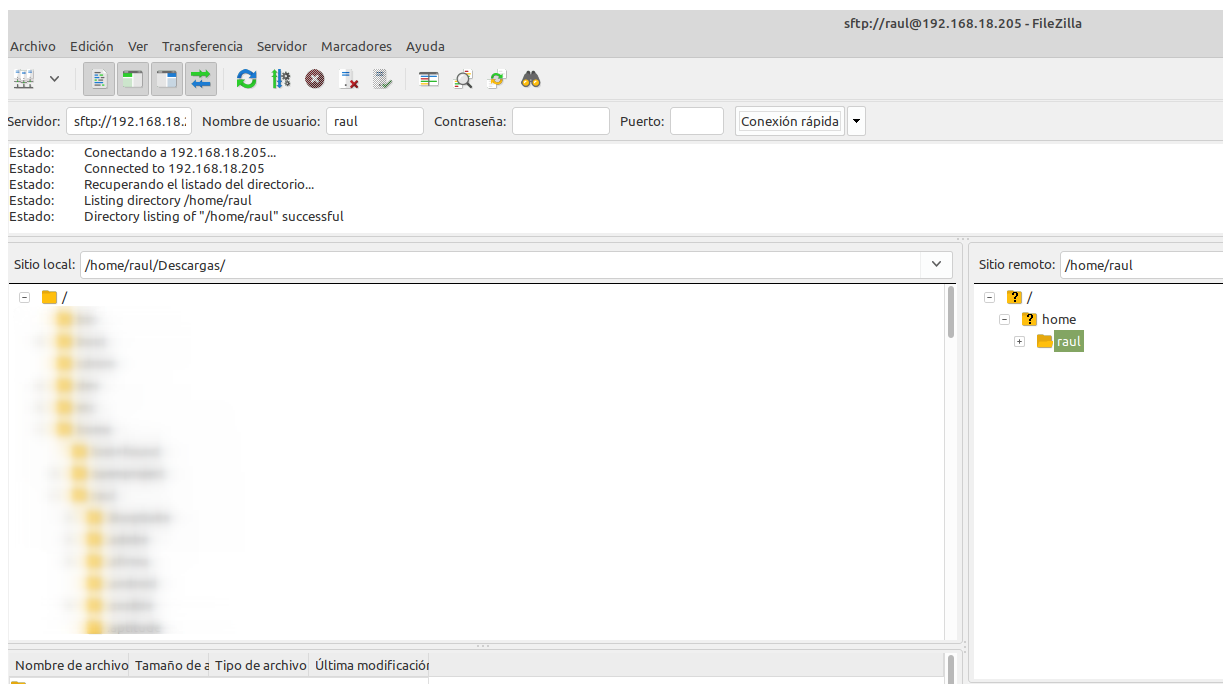
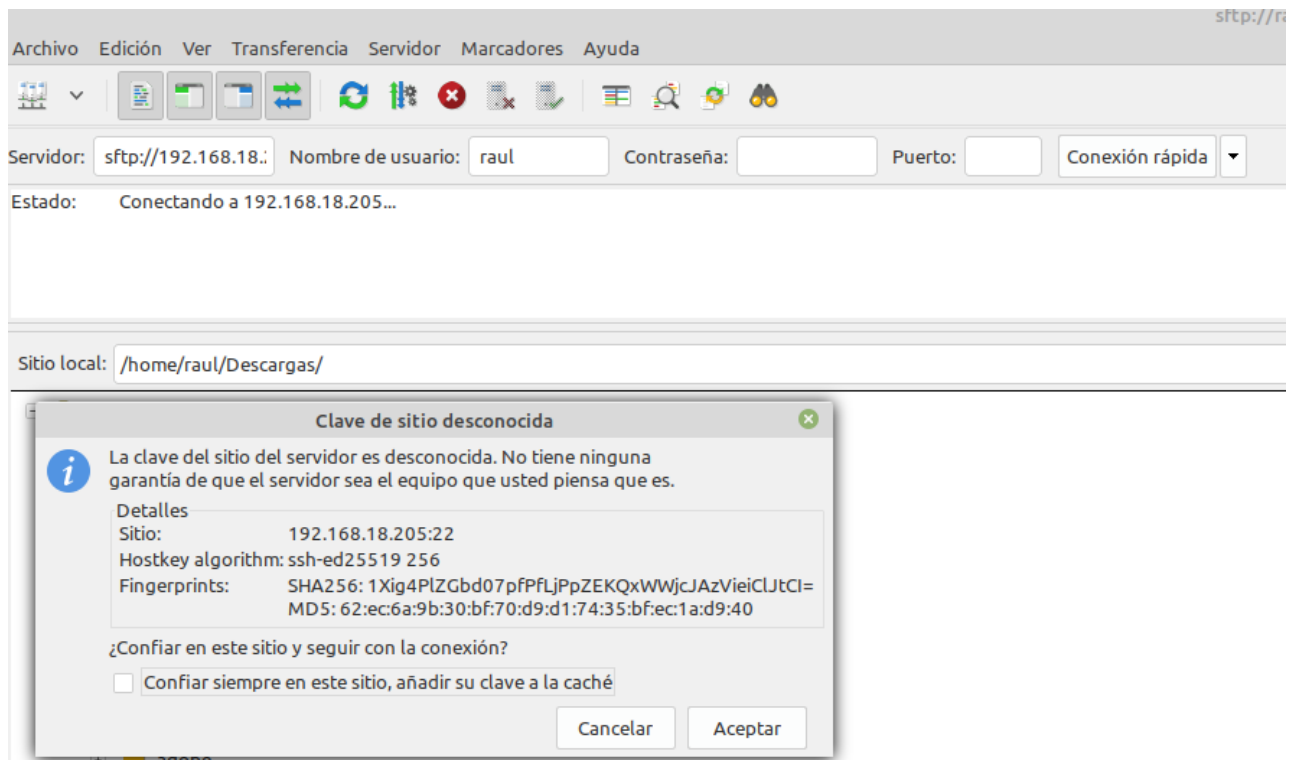
Si lo que quisiéramos es conectarnos por **SFTP**, exactamente igual de válido, haríamos:



Fijaos que al utilizar las claves de SSH que ya estamos utilizando desde la Práctica 1, no se debe introducir la contraseña, únicamente el nombre de usuario.

Puesto que nos estamos conectando usando las claves FTP, nos sale el mismo aviso que nos salía al conectarnos por primera vez por SSH a

nuestra Debian, que aceptamos porque sabemos que no entraña ningún peligro en este caso:



Y vemos que al ser una especie de conexión SSH, nos conecta al home del usuario, en lugar de a la carpeta ftp. A partir de aquí ya procederíamos igual que en el otro caso.

Recordemos que debemos tener nuestro sitio web en la carpeta `/var/www` y darle los permisos adecuados, de forma similar a cómo hemos hecho con el otro sitio web.

El comando que nos permite descomprimir un `.zip` en un directorio concreto es:

```
unzip archivo.zip -d /nombre/directorio
```

Si no tuvieráis `unzip` instalado, lo instaláis:

```
sudo apt-get update && sudo apt-get install unzip
```

Cuestiones finales

Cuestión 1

¿Qué pasa si no hago el link simbólico entre `sites-available` y `sites-enabled` de mi sitio web?

Cuestión 2

¿Qué pasa si no le doy los permisos adecuados a `/var/www/nombre_web`?

Evaluación

Criterio

Configuración correcta del servidor web **1 pto**

Comprobación del correcto funcionamiento del primer sitio web **3 ptos**

Configuración correcta y comprobación del funcionamiento de una segunda web **2 ptos**

Cuestiones finales **2 ptos**

Se ha prestado especial atención al formato del documento, utilizando la plantilla actualizada y haciendo un correcto uso del lenguaje técnico. **2 ptos**

Práctica 2.2 – Autenticación en Nginx

Requisitos antes de comenzar la práctica

Atención, muy importante antes de empezar

- La práctica 2.1 ha de estar funcionando correctamente
- No empezar la práctica antes de tener la 2.1 **funcionando y comprobada**

Introducción

En el contexto de una transacción HTTP, la autenticación de **acceso básica** es un método diseñado para permitir a un navegador web, u otro programa cliente, proveer credenciales en la forma de usuario y contraseña cuando se le solicita una página al servidor.

La autenticación básica, como su nombre lo indica, es la forma más básica de autenticación disponible para las aplicaciones Web. Fue definida por primera vez en la especificación HTTP en sí y no es de ninguna manera elegante, pero cumple su función.

Este tipo de autenticación es el tipo más simple disponible pero adolece de importantes problemas de seguridad que no la hacen recomendable en muchas situaciones. No requiere el uso ni de cookies, ni de identificadores de sesión, ni de página de ingreso.

Paquetes necesarios

Para esta práctica podemos utilizar la herramienta openssl para crear las contraseñas.

En primer lugar, debemos comprobar si el paquete está instalado:

```
dpkg -l | grep openssl
```

Y si no lo estuviera, instalarlo.

Creación de usuarios y contraseñas para el acceso web

Crearemos un archivo oculto llamado “.htpasswd” en el directorio de configuración /etc/nginx donde guardar nuestros usuarios y contraseñas (la -c es para crear el archivo):

```
sudo sh -c "echo -n 'vuestro_nombre:' >> /etc/nginx/.htpasswd"
```

Ahora crearemos un password cifrado para el usuario:

```
sudo sh -c "openssl passwd -apr1 >> /etc/nginx/.htpasswd"
```

Este proceso se podrá repetir para tantos usuarios como haga falta.

- Crea dos usuarios, uno con tu nombre y otro con tu primer apellido
- Comprueba que el usuario y la contraseña aparecen cifrados en el fichero:
-

```
cat /etc/nginx/.htpasswd
```

Configurando el servidor Nginx para usar autenticación básica

Editaremos la configuración del server block sobre el cual queremos aplicar la restricción de acceso. Utilizaremos para esta autenticación el sitio web de *Perfect Learn*:

Info

Recuerda que un *server block* es cada uno de los dominios (server {...} dentro del archivo de configuración) de alguno de los sitios web que hay en el servidor.



```
sudo nano /etc/nginx/sites-available/nombre_web
```

Debemos decidir qué recursos estarán protegidos. Nginx permite añadir restricciones a nivel de servidor o en un location (directorio o archivo) específico. Para nuestro ejemplo, vamos a proteger el document root (la raíz, la página principal) de nuestro sitio.

Utilizaremos la directiva `auth_basic` dentro del location y le pondremos el nombre a nuestro dominio que será mostrado al usuario al solicitar las credenciales. Por último, configuramos Nginx para que utilice el fichero que previamente hemos creado con la directiva `auth_basic_user_file` :

```
server {
    listen 80;
    listen [::]:80;

    root /var/www/webraul/html/simple-static-website;
    index index.html index.htm index.nginx-debian.html;

    server_name nombre_web;

    location / {
        auth_basic "Área restringida";
        auth_basic_user_file /etc/nginx/.htpasswd;
        try_files $uri $uri/ =404;
    }
}
```

Una vez terminada la configuración, reiniciamos el servicio para que aplique nuestra política de acceso.

```
sudo systemctl restart nginx
```

Probando la nueva configuración

Comprobación 1

Comprueba desde tu máquina física/anfitrión que puedes acceder a `http://nombre-sitio-web` y que se te solicita autenticación

Comprobación 2

Comprueba que, si decides cancelar la autenticación, se te negará el acceso al sitio con un error. ¿Qué error es?

Cuidado

Una vez os autenticáis con éxito, el navegador guardará esta autenticación exitosa y no volverá a pedir os *usuario/contraseña*.

Llegados a ese punto, si queréis volver a probar a autenticaros, tendréis que abriros una **Nueva ventana privada** del navegador.

Tareas

Tarea 1

- Intenta entrar primero con un usuario erróneo y luego con otro correcto. Puedes ver todos los sucesos y registros en los logs `access.log` y `error.log`
- Adjunta una captura de pantalla de los logs donde se vea que intentas entrar primero con un usuario inválido y con otro válido. Indica dónde podemos ver los errores de usuario inválido o no encontrado, así como donde podemos ver el número de error que os aparecía antes

Cuando hemos configurado el siguiente bloque:

```
location / {  
    auth_basic "Àrea restringida";  
    auth_basic_user_file /etc/nginx/.htpasswd;  
    try_files $uri $uri/ =404;  
}
```

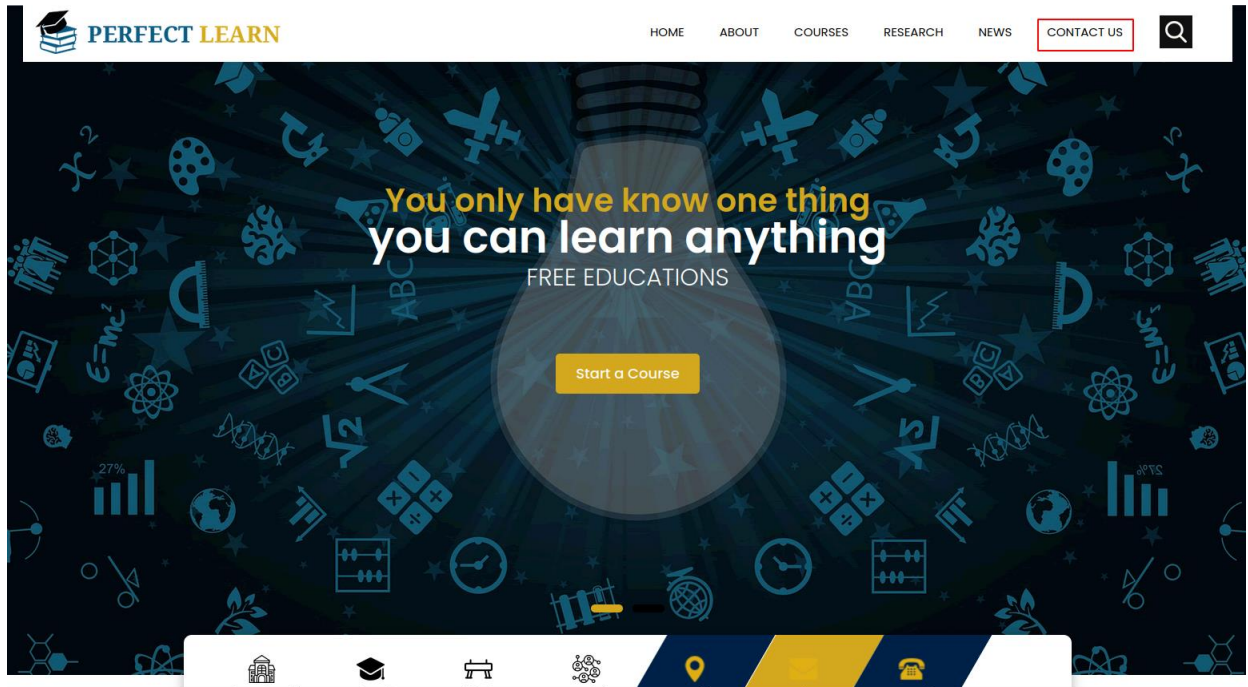
La autenticación se aplica al directorio/archivo que le indicamos en la declaración del `location` y que en este caso el raíz `/`.

Así pues, esta restricción se aplica al directorio raíz o base donde residen los archivos del sitio web y que es:

`/var/www/webraul/html/simple-static-website`

Y a todos los archivos que hay dentro, ya que no hemos especificado ninguno en concreto.

Ahora bien, vamos a probar a aplicar autenticación sólo a una parte de la web. Vamos a intentar que sólo se necesite autenticación para entrar a la parte de portfolio:



Esta sección se corresponde con el archivo `contact.html` dentro del directorio raíz.

Tarea 2

Borra las dos líneas que hacen referencia a la autenticación básica en el location del directorio raíz. Tras ello, añade un nuevo location debajo con la autenticación básica para el archivo/sección `contact.html` únicamente.

Warning

Fijaos que debéis tener cuidado porque la última línea del archivo ha de ser `}` que cierra la primera línea `server {` del archivo.

Combinación de la autenticación básica con la restricción de acceso por IP

La autenticación básica HTTP puede ser combinada de forma efectiva con la restricción de acceso por dirección IP. Se pueden implementar dos escenario:

- Un usuario debe estar ambas cosas, autenticado y tener una IP válida
- Un usuario debe o bien estar autenticado, o bien tener una IP válida

Veamos cómo lo haríamos:

1. Como permitir o denegar acceso sobre una IP concreta (directivas **allow** y **deny**, respectivamente). Dentro del block server o archivo de configuración del dominio web, que recordad está en el directorio sites-available:

```
location /api {  
    #...  
    deny 192.168.1.2;  
    allow 192.168.1.1/24;  
    allow 127.0.0.1;  
    deny all;  
}
```

El acceso se garantizará ala IP 192.168.1.1/24, excluyendo a la dirección 192.168.1.2.

Hay que tener en cuenta que las directivas allow y deny se irán aplicando en el orden en el que aparecen el archivo.

Aquí aplican sobre la location /api (esto es sólo un ejemplo de un hipotético directorio o archivo), pero podrían aplicar sobre cualquiera, incluida todo el sitio web, la location raíz /.

La última directiva deny all quiere decir que por defecto denegaremos el acceso a todo el mundo. Por eso hay que poner los allow y deny más específicos justo antes de esta, porque al evaluarse en orden de aparición, si los pusiéramos debajo se denegaría el acceso a todo el mundo, puesto que deny all sería lo primero que se evaluaría.

2. Combinar la restricción IP y la autenticación HTTP con la directiva **satisfy**.

Si establecemos el valor de la directiva a "all", el acceso se permite si el cliente satisface ambas condiciones (IP y usuario válido). Si lo establecemos a "any", el acceso se permite si se satisface al menos una de las dos condiciones.

```
location /api {  
    #...  
    satisfy all;  
  
    deny 192.168.1.2;  
    allow 192.168.1.1/24;  
    allow 127.0.0.1;  
    deny all;  
  
    auth_basic "Administrator's Area";  
    auth_basic_user_file conf/htpasswd;  
}
```

Tareas

Tarea 1

Configura Nginx para que no deje acceder con la IP de la máquina anfitriona al directorio raíz de una de tus dos webs. Modifica su server block o archivo de configuración. Comprueba como se deniega el acceso:

- Muestra la página de error en el navegador
- Muestra el mensaje de error de error.log

Tarea 2

Configura Nginx para que desde tu máquina anfitriona se tenga que tener tanto una IP válida como un usuario válido, ambas cosas a la vez, y comprueba que sí puede acceder sin problemas

Cuestiones finales

Cuestión 1

Supongamos que yo soy el cliente con la IP 172.1.10.15 e intento acceder al directorio `web_muy_guay` de mi sitio web, equivocándome al poner el usuario y contraseña. ¿Podré acceder? ¿Por qué?

```
location /web_muy_guay {
#...
satisfy all;
deny 172.1.10.6;
allow 172.1.10.15;
allow 172.1.3.14;
deny all;
auth_basic "Cuestión final 1";
auth_basic_user_file conf/htpasswd;
}
```

Cuestión 2

ask "Cuestión 1" Supongamos que yo soy el cliente con la IP 172.1.10.15 e intento acceder al directorio `web_muy_guay` de mi sitio web, introduciendo correctamente usuario y contraseña. ¿Podré acceder? ¿Por qué?

```
location /web_muy_guay {
#...
satisfy all;
deny all;
deny 172.1.10.6;
allow 172.1.10.15;
allow 172.1.3.14;

auth_basic "Cuestión final 2: The revenge";
auth_basic_user_file conf/htpasswd;
}
```

Cuestión 3

Supongamos que yo soy el cliente con la IP 172.1.10.15 e intento acceder al directorio `web_muy_guay` de mi sitio web, introduciendo correctamente usuario y contraseña. ¿Podré acceder? ¿Por qué?

```
location /web_muy_guay {
#...
satisfy any;
deny 172.1.10.6;
deny 172.1.10.15;
allow 172.1.3.14;

auth_basic "Cuestión final 3: The final combat";
auth_basic_user_file conf/htpasswd;
}
```

Cuestión 4

A lo mejor no sabéis que tengo una web para documentar todas mis excursiones espaciales con Jeff, es esta: [Jeff Bezos y yo](#)

Supongamos que quiero restringir el acceso al directorio de proyectos porque es muy secreto, eso quiere decir añadir autenticación básica a la URL: [Proyectos](#)

Completa la configuración para conseguirlo:

```
server {  
    listen 80;  
    listen [::]:80;  
    root /var/www/freewebsitetemplates.com/preview/space-science;  
    index index.html index.htm index.nginx-debian.html;  
    server_name freewebsitetemplates.com www.freewebsitetemplates.com;  
    location / {  
        try_files $uri $uri/ =404;  
    }  
}
```

Evaluación

Criterio

Puntuación

Configuración correcta de la autorización básica de Nginx, comprobación e identificación del error **2 puntos**

Capturas correctas del log **1 puntos**

Configuración correcta de la autorización básica en contact **1.5 puntos**

Correcta configuración y comprobación de las tareas de autenticación básica y restricción por IP **1.5 puntos**

Cuestiones finales **2 puntos**

Se ha prestado especial atención al formato del documento, utilizando la plantilla actualizada y haciendo un correcto uso del lenguaje técnico **2 puntos**

Práctica 2.3 – Proxy inverso con Nginx

Requisitos antes de comenzar la práctica

Atención, importante antes de comenzar

- La práctica 2.1 ha de estar funcionando correctamente
- No comenzar la práctica antes de tener la 2.1 **funcionando y comprobada**

Introducción

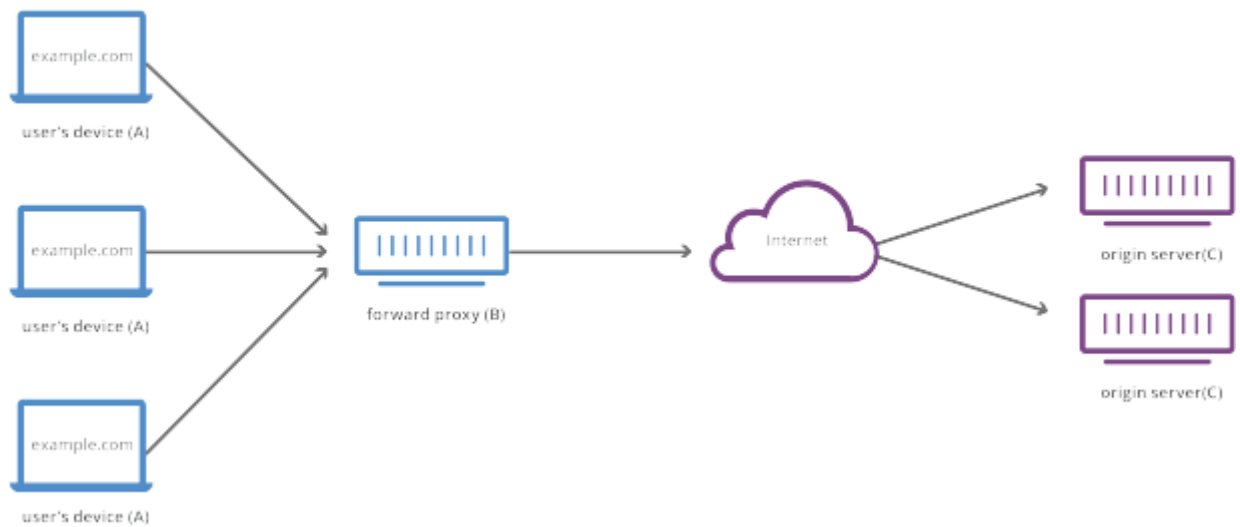
¿Qué es un servidor proxy?

Un proxy de reenvío, a menudo llamado proxy, servidor proxy o proxy web, es un servidor que se encuentra frente a un grupo de máquinas cliente. Cuando esas máquinas realizan solicitudes a sitios y servicios en Internet, el servidor proxy intercepta esas solicitudes y luego se comunica con los servidores web en nombre de esos clientes, como un intermediario.

Por ejemplo, tomemos como ejemplo 3 máquinas involucradas en una comunicación típica de proxy de reenvío:

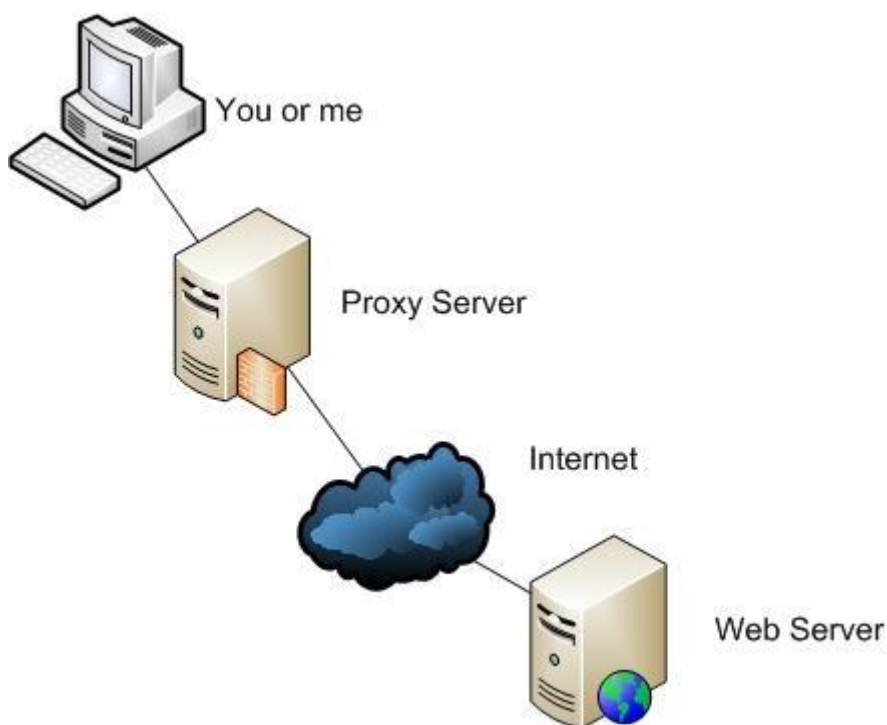
- A: Esta es la máquina del hogar de un usuario.
- B: este es un servidor proxy de reenvío
- C: este es el servidor de origen de un sitio web (donde se almacenan los datos del sitio web)

Forward Proxy Flow



En una comunicación estándar por Internet, la máquina A se comunicaría directamente con la máquina C, con el cliente enviando solicitudes al servidor de origen y el servidor de origen respondiendo al cliente. Cuando hay un proxy de reenvío, A enviará solicitudes a B, que luego reenviará la solicitud a C. C enviará una respuesta a B, que reenviará la respuesta a A.

¿Por qué agregar este intermediario adicional a nuestra actividad en Internet?



Hay algunas razones por las que uno podría querer usar un proxy de reenvío:

- ***Para evitar restricciones de navegación estatales o institucionales:*** algunos gobiernos, escuelas y otras organizaciones usan firewalls para dar a sus usuarios acceso a una versión limitada de Internet. Se puede usar un proxy de reenvío para sortear estas restricciones, ya que permiten que el usuario se conecte al proxy en lugar de directamente a los sitios que está visitando.
- ***Para bloquear el acceso a cierto contenido:*** a la inversa, los proxies también se pueden configurar para bloquear el acceso de un grupo de usuarios a ciertos sitios. Por ejemplo, una red escolar puede estar configurada para conectarse a la web a través de un proxy que habilita reglas de filtrado de contenido, negándose a reenviar respuestas de Facebook y otros sitios de redes sociales.
- ***Para proteger su identidad en línea:*** en algunos casos, los usuarios habituales de Internet simplemente desean un mayor anonimato en línea, pero en otros casos, los usuarios de Internet viven en lugares donde el gobierno puede imponer graves consecuencias a los disidentes políticos. Criticar al gobierno en un foro web o en las redes sociales puede dar lugar a multas o encarcelamiento para estos usuarios. Si uno de estos disidentes usa un proxy de reenvío para conectarse a un sitio web donde publica comentarios políticamente sensibles, la dirección IP utilizada para publicar los comentarios será más difícil de rastrear hasta el disidente. Solo estará visible la dirección IP del servidor proxy.

¿En qué se diferencia un proxy inverso?

Estaríamos hablando del caso opuesto al anterior.

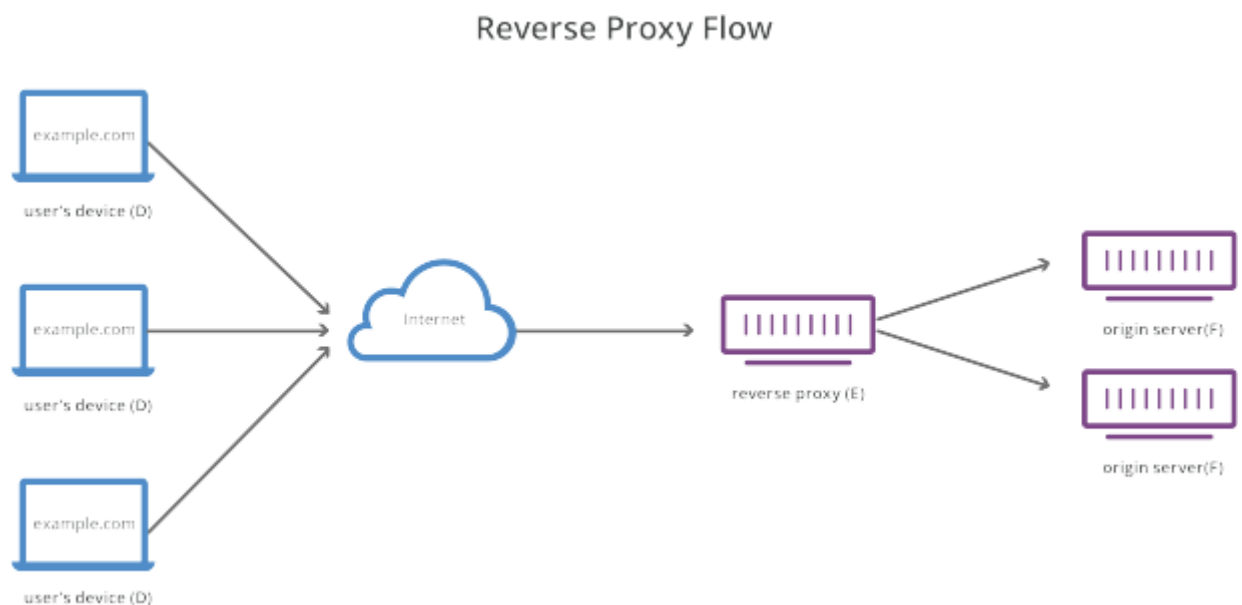
Un proxy inverso es un servidor que se encuentra frente a uno o más servidores web, interceptando las solicitudes de los clientes. Esto es diferente de un proxy de reenvío, donde el proxy se encuentra frente a los clientes. Con un proxy inverso, cuando los clientes envían solicitudes al servidor de un sitio web, esas solicitudes son interceptadas en la frontera de la red por el servidor proxy inverso. El servidor proxy inverso enviará solicitudes y recibirá respuestas del servidor del sitio web.

La diferencia entre un proxy directo e inverso es sutil pero importante. Una forma simplificada de resumir sería decir que un **proxy de reenvío se encuentra frente a un cliente y garantiza que ningún servidor de origen se comunique nunca directamente con ese cliente específico**. Por otro lado, un **proxy inverso se encuentra frente a un servidor de origen y garantiza**

que ningún cliente se comuniquen nunca directamente con ese servidor de origen.

Una vez más, ilustremos nombrando las máquinas involucradas:

- D: cualquier número de ordenadores domésticos de los usuarios
- E: este es un servidor proxy inverso
- F: uno o más servidores de origen



Normalmente, todas las solicitudes de D irían directamente a F, y F enviaría respuestas directamente a D. Con un proxy inverso, todas las solicitudes de D irán directamente a E, y E enviará sus solicitudes a F y recibirá respuestas de F. E luego transmita las respuestas apropiadas a D.

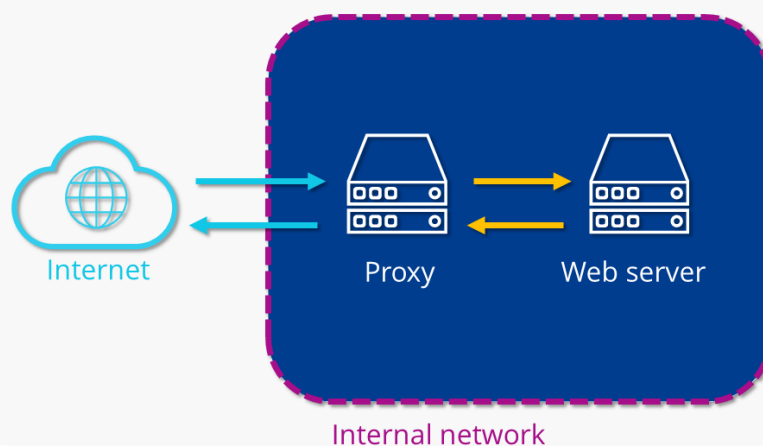
A continuación se describen algunos de los beneficios de un proxy inverso:

- **Balanceo de carga:** es posible que un sitio web popular que recibe millones de usuarios todos los días no pueda manejar todo el tráfico entrante del sitio con un solo servidor de origen. En cambio, el sitio se puede distribuir entre un grupo de servidores diferentes, todos manejando solicitudes para el mismo sitio. En este caso, un proxy inverso puede proporcionar una solución de balanceo de carga que distribuirá el tráfico entrante de manera uniforme entre los diferentes servidores para evitar que un solo servidor se sobrecargue. En el caso de

que un servidor falle por completo, otros servidores pueden intensificar para manejar el tráfico.

- **Protección contra ataques:** con un proxy inverso en su lugar, un sitio web o servicio nunca necesita revelar la dirección IP de su (s) servidor (es) de origen. Esto hace que sea mucho más difícil para los atacantes aprovechar un ataque dirigido contra ellos, como un ataque DDoS.
- **Almacenamiento en caché:** un proxy inverso también puede almacenar contenido en caché, lo que resulta en un rendimiento más rápido. Por ejemplo, si un usuario en París visita un sitio web con proxy inverso con servidores web en Los Ángeles, el usuario podría conectarse a un servidor proxy inverso local en París, que luego tendrá que comunicarse con un servidor de origen en Los Ángeles. El servidor proxy luego puede almacenar en caché (o guardar temporalmente) los datos de respuesta. Los usuarios parisinos posteriores que naveguen por el sitio obtendrán la versión en caché local del servidor proxy inverso parisino, lo que dará como resultado un rendimiento mucho más rápido.
- **Cifrado SSL** - Cifrado y descifrado SSL (o TLS comunicaciones) para cada cliente pueden ser computacionalmente caro para un servidor de origen. Se puede configurar un proxy inverso para descifrar todas las solicitudes entrantes y cifrar todas las respuestas salientes, liberando valiosos recursos en el servidor de origen.

Reverse proxy



Tarea

Configuraciones

Nginx servidor web

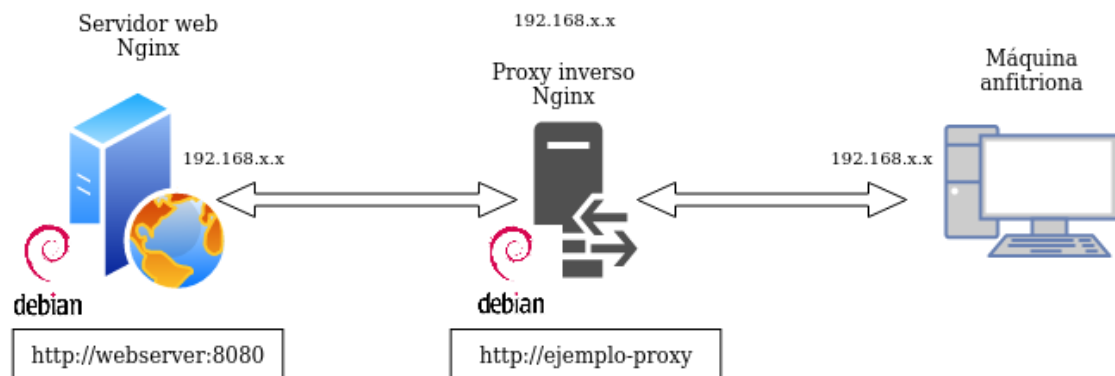
Vamos a configurar dos Debian con sendos servidores Nginx. Tenéis la máquina virtual inicial y debéis clonarla para tener una segunda:

- Uno servirá las páginas web que ya hemos configurado, así pues, utilizaremos el servidor que ya tenemos configurado de la Práctica 2.1.
- El nuevo servidor clon Debian con Nginx configurado como proxy inverso
- Realizaremos las peticiones HTTP desde el navegador web de nuestra máquina física/anfitrión hacia el proxy clonado, que nos redirigirá al servidor web original

Cuidado

Ojo al clonar las máquinas virtuales porque hay que darle a crear una nueva MAC, de lo contrario no tendréis IP en esa máquina.

El diagrama de red quedaría así:



Para que todo quede más diferenciado y os quede más claro que la petición está pasando por el proxy inverso y llega al servidor web destino, vamos a hacer que cada uno de los servidores escuche las peticiones en un puerto distinto.

1. En primer lugar, debéis cambiar el nombre que tuviera vuestra web por el de webserver, ello implica:
 - Cambiar el nombre del archivo de configuración de sitios disponibles para Nginx

- Cambiar el nombre del sitio web dentro de este archivo de configuración donde haga falta
 - No os olvidéis de eliminar el link simbólico antiguo con el comando `unlink nombre_del_link` dentro de la carpeta `sites-enabled` y crear el nuevo para el nuevo nombre de archivo.
2. En el archivo de configuración del sitio web, en lugar de hacer que el servidor escuche en el puerto 80, cambiadlo al 8080.
 3. Reiniciar Nginx

Nginx proxy inverso

Ahora, cuando intentamos acceder a `http://ejemplo-proxy` (o el nombre que tuvieráis de vuestra web de las prácticas anteriores), en realidad estaremos accediendo al proxy, que nos redirigirá a `http://webserver:8080`, el servidor web que acabamos de configurar para que escuche con ese nombre en el puerto 8080.

Para ello:

- Crear un archivo de configuración en `sites-available` con el nombre `ejemplo-proxy` (o el que tuvierais vosotros)
- Este archivo de configuración será más simple, tendrá la siguiente forma

```
server {
listen __;
server_name _____;
    location / {
proxy_pass http://_____:__;
    }
}
```

Donde, ***mirando el diagrama de red y teniendo en cuenta la configuración hecha hasta ahora***, debéis completar:

- El puerto donde está escuchando el proxy inverso
- El nombre de vuestro dominio o sitio web original al que accedemos en el proxy
- La directiva `proxy_pass` indica a dónde se van a redirigir las peticiones, esto es, al servidor web. Por tanto, debéis poner la IP y número de puerto adecuados de vuestro sitio web configurado en el apartado anterior.
- Crear el link simbólico pertinente

Esto es para simular la situación en la que nosotros, como clientes, cuando accedamos a nuestro sitio web, no necesitemos saber cómo está todo configurado, sólo necesitamos saber el nombre de la web.

¡Atención, muy importante!

Debéis modificar el archivo host que configurastéis en la práctica 2.1. Si miráis el diagrama de red, ahora el nombre de vuestro sitio web se corresponderá con la IP de la nueva máquina clon que hace de proxy. Será ésta la encargada de redirigirnos automáticamente al verdadero sitio web.

Comprobaciones

Si accedéis a vuestro sitio web, debéis poder seguir accediendo sin problemas.

- Comprobad en los access.log de los dos servidores que llega la petición
- Comprobad además la petición y respuesta con las herramientas de desarrollador de Firefox en Xubuntu. Pulsando F12 en el navegador os aparecerán estas herramientas

The screenshot shows the Firefox Developer Tools interface with the Network tab selected. A red box highlights the 'Red' button in the top toolbar. Another red box highlights the first request in the list, a GET request to 'http://ejemplo1/'. The details pane on the right shows the response status as '200 OK' and lists the request headers, including 'Accept', 'Accept-Encoding', 'Accept-Language', 'Cache-Control', 'Connection', 'Host', 'Pragma', 'Upgrade-Insecure-Requests', and 'User-Agent'.

Estado	Método	Domini	Archivo	Indicador	Tipo	Transferido	Tama...
200	GET	ejemplo1	/		html	2,99 KB	8,47 KB
200	GET	ejemplo1	1.jpg		img	59,94 KB	59,70 ...
200	GET	ejemplo1	2.jpg		img	67,07 KB	66,82 ...
200	GET	ejemplo1	3.jpg		img	93,04 KB	92,79 ...
404	GET	ejemplo1	apple-touch-icon-114x114.png		FaviconLoader...	318 B	162 B
200	GET	ejemplo1	bg.png		img	17,50 KB	17,26 ...
200	GET	ejemplo1	bodybg.png		img	81,70 KB	81,45 ...
200	GET	ejemplo1	bootstrap-responsive.css		stylesheet	2,55 KB	2,31 KB
200	GET	ejemplo1	bootstrap.css		stylesheet	56,96 KB	56,72 ...
200	GET	ejemplo1	bootstrap.js		script	55,20 KB	54,85 ...
200	GET	ejemplo1	custom.css		stylesheet	20,51 KB	20,27 ...
200	GET	ejemplo1	facebook.png		img	3,95 KB	3,71 KB
404	GET	ejemplo1	favicon.ico		FaviconLoader...	318 B	162 B
200	GET	ejemplo1	flickr.png		img	4,19 KB	3,94 KB
200	GET	ejemplo1	formvalidation.js		script	2,36 KB	2,11 KB
200	GET	ejemplo1	google.png		img	4,48 KB	4,24 KB
200	GET	ejemplo1	hireme.png		img	4,45 KB	4,21 KB
200	GET	ejemplo1	html5placeholder.jquery.js		script	3,48 KB	3,23 KB
200	GET	ejemplo1	jquery.js		script	92,87 KB	92,62 ...
200	GET	ejemplo1	jquery.tweet.js		script	14,51 KB	14,25 ...
200	GET	ejemplo1	logo.png		img	11,70 KB	11,46 ...

23 solicitudes 606,39 KB / 606,14 KB transferido Finalizado: 868 ms DOMContentLoaded: 260 ms load: 734 ms

En la primera petición (marcada en rojo), utilizando el apartado “Red” (también marcado en rojo) y también en rojo está señalado dónde se puede ver la respuesta de la petición GET HTTP (200 OK).

También vemos las cabeceras que se incluyen en la petición (método GET) y en la respuesta a esta petición.

Añadiendo cabeceras

Además de haber mirado los logs, vamos a demostrar aún de forma más clara que la petición está pasando por el proxy inverso y que está llegando al servidor web y que vuelve por el mismo camino.

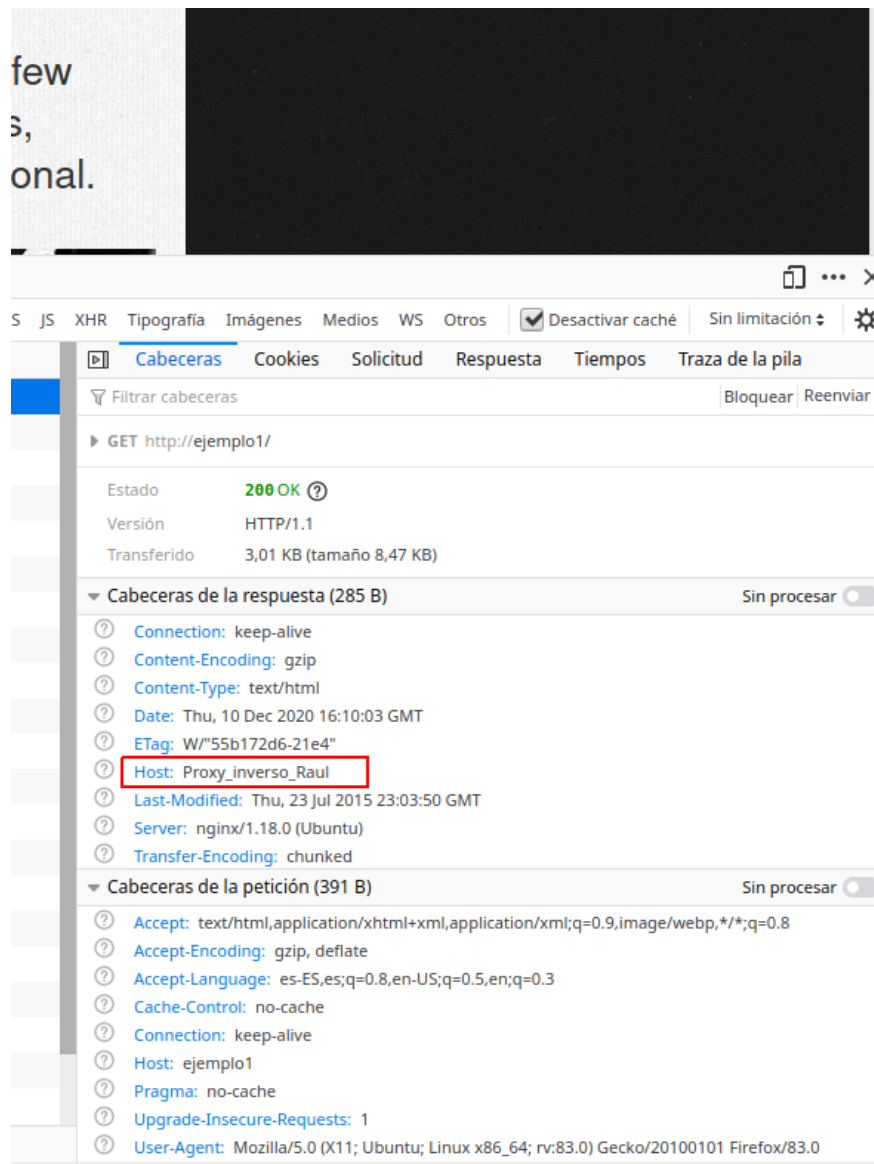
Si recordáis de teoría, el servidor web es capaz de añadir cabeceras en las respuestas a las peticiones.

Así pues, vamos a configurar tanto el proxy inverso como el servidor web para que añadan cada uno la cabecera "Host" que también vimos en teoría.

Para añadir cabeceras, en el archivo de configuración del sitio web debemos añadir dentro del bloque `location / { ... }` debemos añadir la directiva:

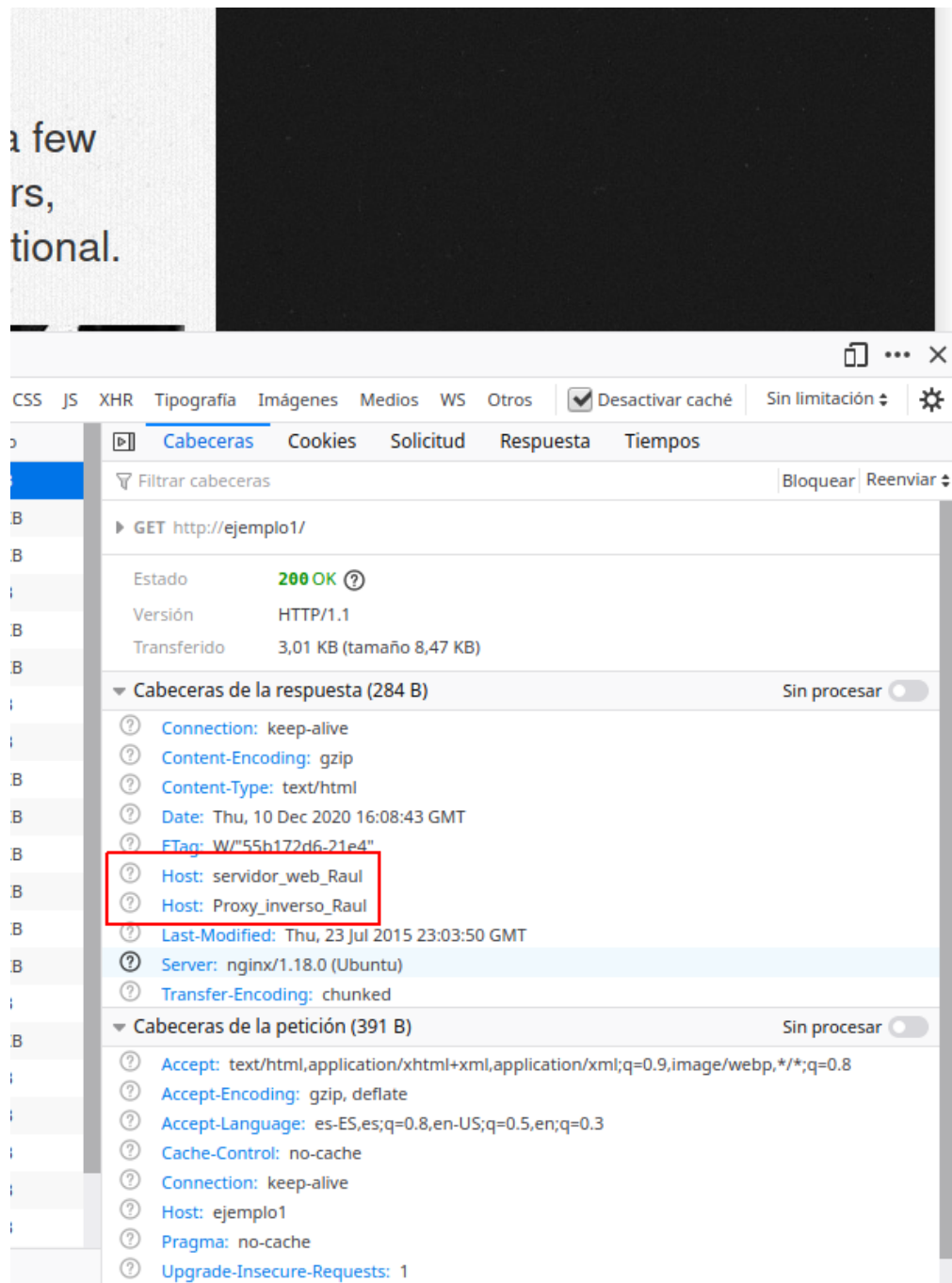
```
add_header Host nombre_del_host;
```

1. Añadiremos primero esta cabecera únicamente en el archivo de configuración del sitio web del proxy inverso. El `Nombre_del_host` será `Proxy_inverso_vuestronombre`.
2. Reiniciamos Nginx
3. Comprobamos que podemos acceder al sitio web sin problemas
4. Con las herramientas de desarrollador comprobamos que la petición ha pasado por el proxy inverso que ha añadido la cabecera en la respuesta:

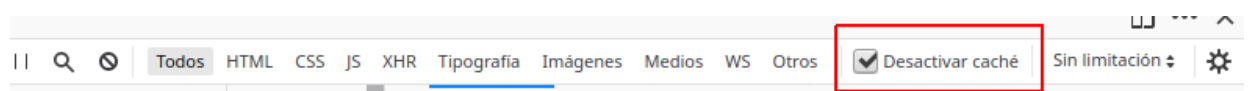


5. Hacemos lo propio con el servidor web. Esta vez el Nombre_del_host será servidor_web_vuestronombre.

Si todo está configurado correctamente, al examinar las peticiones y respuestas, os aparecerán las dos cabeceras que han incluido en la respuesta tanto el proxy inverso como el servidor web. .



Es muy importante que para realizar estas comprobaciones tengáis marcado el checkbox *Desactivar caché* o en una ventana privada del navegador.



Si no marcáis esto, la página se guardará en la memoria caché del navegador y no estaréis recibiendo la respuesta del servidor sino de la caché del navegador, lo que puede dar lugar a resultados erróneos.

Evaluación

Criterio

Configuración correcta y completa del servidor web **3 ptos**

Configuración correcta y completa del proxy inverso **4 ptos**

Comprobación del correcto funcionamiento, incluyendo las cabeceras **1 pto**

Se ha prestado especial atención al formato del documento, utilizando la plantilla actualizada y haciendo un correcto uso del lenguaje técnico **2 ptos**

Práctica 2.4 – Balanceo de carga con proxy inverso en Nginx

Requisitos antes de comenzar la práctica

Atención, muy importante antes de empezar

- La práctica 2.3 debe estar funcionando correctamente
- No empezar la práctica antes de tener la 2.3 funcionando y **comprobada**

Introducción

Los servidores proxy inversos y los balanceadores de carga son componentes de una arquitectura informática cliente-servidor. Ambos actúan como intermediarios en la comunicación entre los clientes y los servidores, realizando funciones que mejoran la eficiencia.

Las definiciones básicas son simples:

- Un [proxy inverso](#) acepta una solicitud de un cliente, la reenvía a un servidor que puede cumplirla y devuelve la respuesta del servidor al cliente.
- Un [balanceador de carga](#) distribuye las solicitudes entrantes del cliente entre un grupo de servidores, en cada caso devolviendo la respuesta del servidor seleccionado al cliente apropiado.

Suenan bastante similares, ¿verdad? Ambos tipos de aplicaciones se ubican entre clientes y servidores, aceptando solicitudes del primero y entregando respuestas del segundo. No es de extrañar que haya confusión sobre qué es un proxy inverso y un balanceador de carga. Para ayudar a diferenciarlos, exploremos cuándo y por qué normalmente se implementan en un sitio web.

Proxy inverso

Ya conocemos este concepto de la práctica anterior.

Mientras que implementar un balanceador de carga solo tiene sentido cuando se tienen varios servidores, a menudo tiene sentido implementar

un proxy inverso incluso con un solo servidor web o servidor de aplicaciones.

Se puede pensar en el proxy inverso como la "cara pública" de un sitio web. Su dirección es la que se anuncia para el sitio web y se encuentra en la frontera de la red del sitio para aceptar solicitudes de navegadores web y aplicaciones móviles para el contenido alojado en el sitio web.

Balanceadores de carga

Los balanceadores de carga se implementan con mayor frecuencia cuando un sitio necesita varios servidores porque el volumen de solicitudes es demasiado para que un solo servidor lo maneje de manera eficiente.

La implementación de varios servidores también elimina un solo punto de fallo, lo que hace que el sitio web sea más confiable. Por lo general, todos los servidores alojan el mismo contenido, y el trabajo del balanceador de carga es distribuir la carga de trabajo de manera que se haga el mejor uso de la capacidad de cada servidor, evite la sobrecarga en cualquiera de ellos y dé como resultado la respuesta más rápida posible al cliente. .

Un balanceador de carga también puede mejorar la experiencia del usuario al reducir la cantidad de respuestas de error que ve el cliente. Lo hace detectando cuándo los servidores caen y desviando las solicitudes de ellos a los otros servidores del grupo. En la implementación más simple, el balanceador de carga detecta el estado del servidor al interceptar las respuestas de error a las solicitudes regulares.

En esta práctica tendremos el escenario donde Nginx hará tanto de proxy inverso como de balanceador de carga al mismo tiempo.

Info

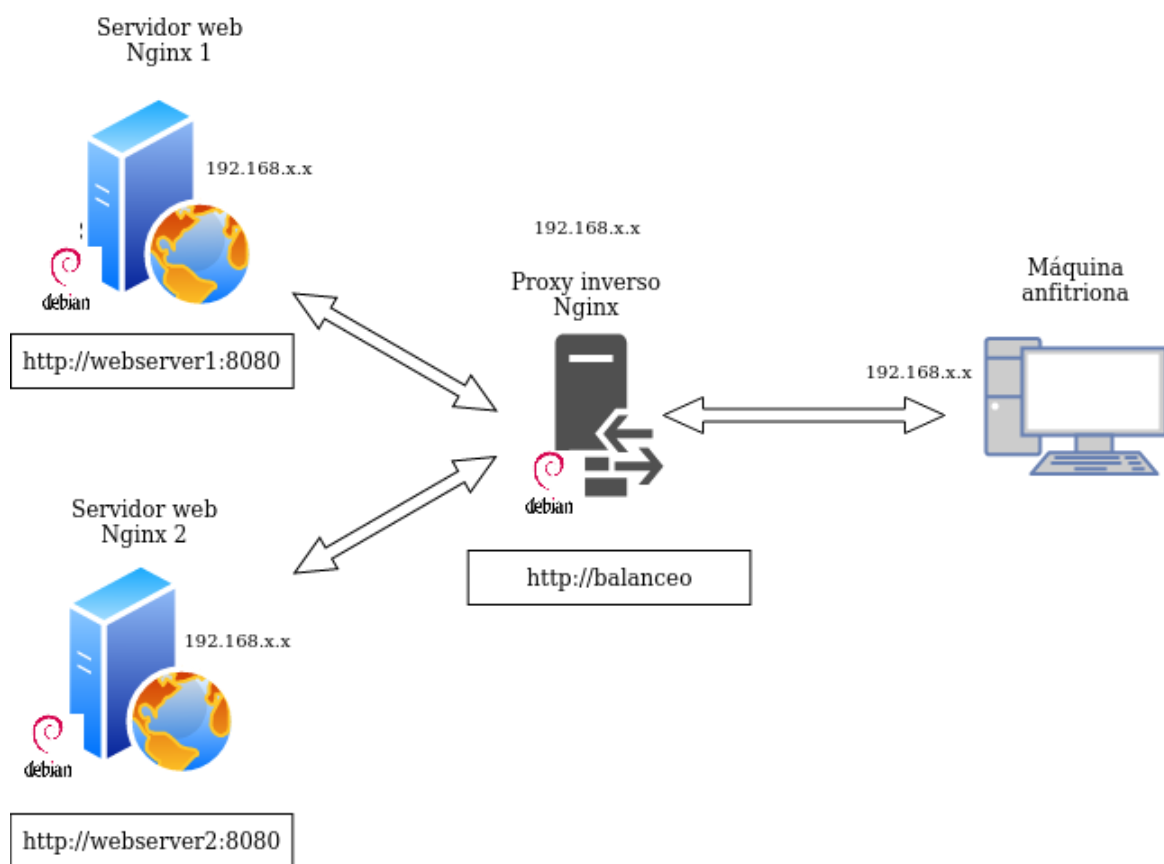
En esta práctica tendremos un escenario donde Nginx hará tanto de proxy inverso como de balanceador de carga al mismo tiempo

Tarea

Vamos a configurar dos servidores web Nginx con dos máquinas Debian, además de reutilizar el proxy inverso Nginx configurado en la práctica anterior. Partiremos por tanto de la configuración de la práctica anterior, añadiendo lo necesario:

- Cada servidor web presentará un sitio web específico para esta práctica
 - El webserver2 debe tener la IP asignada de forma fija mediante la configuración DHCP.
- El proxy inverso que ya teníamos configurado, habrá ahora que configurarlo para que realice el balanceo de carga que deseamos
- Realizaremos las peticiones HTTP desde el navegador web de nuestra máquina anfitriona.

El diagrama de red quedaría así:



Haremos las peticiones web desde el navegador al proxy inverso, que las repartirá entre los dos servidores web que tenemos.

Accederemos a `http://balanceo` y debemos observar que las peticiones, efectivamente, se van repartiendo entre el servidor 1 y el 2.

Configuraciones

Atención

Ya no vamos a utilizar los sitios web que hemos configurado en las prácticas anteriores. Por ello, para evitarnos una serie de problemas que pueden surgir, vamos a desactivarlos.

Dentro de la carpeta `/etc/nginx/sites-enabled` debemos ejecutar `unlink nombre_archivo` para cada uno de los archivos de los sitios web que tenemos.

Si no hacéis esto obtendréis errores en todas las prácticas que quedan de este tema.

Nginx Servidor Web 1

El primer servidor web será el servidor principal que hemos venido utilizando hasta ahora durante el curso, el original, donde tenemos instalado ya el servicio Web.

Debemos configurar este servidor web para que sirva el siguiente `index.html` que debéis crear dentro de la carpeta `/var/www/webserver1/html`:

```
1  <head>
2  |   <title> Prueba de balanceo de carga con Nginx</title>
3  </head>
4  <body>
5  |   <h2>Este es el servidor web 1</h2>
6  |   <p>Comprueba el balanceo de carga con Nginx recargando esta página</p>
7  </body>
8  </html>
9
```

- El nombre del sitio web que debéis utilizar en los archivos correspondientes (*sites-available...*) que debéis crear para Nginx es `webserver1`, así como en sus configuraciones. Fijáos en las configuraciones que hicisteis en prácticas anteriores a modo de referencia.
- El sitio web debe escuchar en el puerto 8080.
- Debéis añadir una cabecera que se llame `Serv_Web1_vuestronombre`.

Nginx Servidor Web 2

Debe ser una máquina Debian, clon del servidor web 1.

En este servidor web debemos realizar una configuración idéntica al servidor web 1 pero cambiando `webserver1` por `webserver2` (también en el `index.html`), así como el nombre de la cabecera añadida, que será `Serv_Web2_vuestronombre`

Warning

Es importante que no quede ninguna referencia a `webserver1` por ningún archivo, de otra forma os dará resultados erróneos y os dificultará mucho encontrar el error.

Nginx Proxy Inverso

Ya disponemos de los dos servidores web entre los que se van a repartir las peticiones que realice el cliente desde el navegador.

Vamos, por tanto, a configurar el proxy inverso para que realice este reparto de peticiones:

- En `sites-available` debéis crear el archivo de configuración con el nombre *balanceo*

Este archivo tendrá el siguiente formato:

```
upstream backend_hosts {
    random;
    server ____:____;
    server ____:____;
}

server {
    listen 80;
    server_name ____;
    location / {
        proxy_pass http://backend_hosts;
    }
}
```

Donde:

- El bloque `upstream` → son los servidores entre los que se va a repartir la carga, que son los dos que hemos configurado anteriormente.
- Si miráis el diagrama y tenéis en cuenta la configuración que habéis hecho hasta ahora, aquí deberéis colocar la IP de cada servidor, así como el puerto donde está escuchando las peticiones web.

- A este grupo de servidores le ponemos un nombre, que es `backend_hosts`

Aclaración

En un sitio web, el backend se encarga de todos los procesos necesarios para que la web funcione de forma correcta. Estos procesos o funciones no son visibles, pero tienen mucha importancia en el buen funcionamiento de un sitio web.

- El parámetro `random` lo que hace es repartir las peticiones HTTP que llegan al proxy inverso de forma completamente aleatoria entre el grupo de servidores que se haya definido en el bloque `upstream` (en nuestro caso sólo hay dos).
 - Pondremos `random` porque es lo más fácil para comprobar que todo funciona bien en la práctica, pero hay diferentes formas de repartir la carga (las peticiones HTTP).

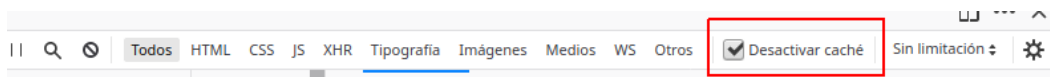
Comprobaciones

Si accedéis a vuestro sitio web, debéis poder seguir accediendo sin problemas.

- Comprobad dándole repetidamente a F5, que accedéis cada vez a uno de los servidores. Se os mostrará el contenido del `index.html` del servidor correspondiente cada vez.
 - Para una doble comprobación, utilizando las herramientas de desarrollador, mostrad que la web que se os muestra coincide con la cabecera que ha añadido el servidor web en la respuesta HTTP.

Recordatorio

Recordad que es muy importante que para realizar estas comprobaciones tengáis marcado el checkbox **Desactivar caché**.



Si no marcáis esto, la página se guardará en la memoria caché del navegador y no estaréis recibiendo la respuesta del servidor sino de la caché del navegador, lo que puede dar lugar a resultados erróneos.

Otra opción, si esto no funcionara, es hacer las pruebas con una nueva ventana privada del navegador.

Comprobación del balanceo de carga cuando cae un servidor

Nuestro balanceador de carga está constantemente monitorizando “la salud” de los servidores web. De esta forma, si uno deja de funcionar por cualquier razón, siempre enviará las solicitudes a los que queden “vivos”. Vamos a comprobarlo:

- Para el servicio Nginx en el servidor web 1 y comprueba, de la misma forma que en el apartado anterior, que todas las solicitudes se envían ahora al servidor web 2
- Tras iniciar de nuevo Nginx en el servidor web 1, repite el proceso con el servidor web 2.

Cuestiones finales

Cuestión 1

Busca información de qué otros métodos de balanceo se pueden aplicar con Nginx y describe al menos 3 de ellos.

Cuestión 2

Si quiero añadir 2 servidores web más al balanceo de carga, describe detalladamente qué configuración habría que añadir y dónde.

Cuestión 3

Describe todos los pasos que deberíamos seguir y configurar para realizar el balanceo de carga con una de las webs de prácticas anteriores.

Indicad la configuración de todas las máquinas (webserver, proxy...) y de sus servicios

Evaluación

Criterio

Configuración correcta y completa del servidor web 1 **2 ptos**

Configuración correcta y completa del servidor web 2n **2 ptos**

Configuración correcta y completa del proxy inverso **3 ptos**

Cuestiones finales **1 pto**

Se ha prestado especial atención al formato del documento, utilizando la plantilla actualizada y haciendo un correcto uso del lenguaje técnico **2 ptos**

Practica 2.5 - Proxy inverso y balanceo de carga con SSL en NGINX

Requisitos antes de comenzar la práctica

Atención, muy importante antes de comenzar

- La práctica 4.4 ha de estar funcionando correctamente
- No comenzar la práctica antes de tener la 1.3 **funcionando y comprobada**

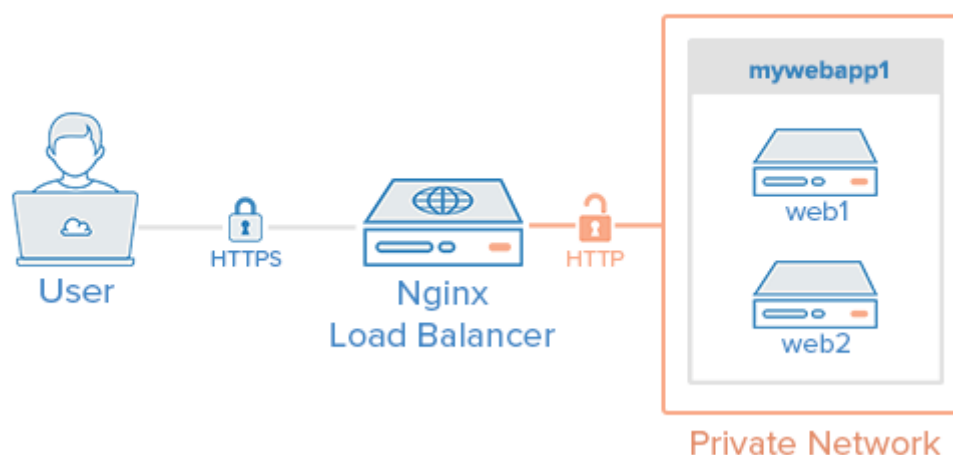
Introducción

A partir de las prácticas anteriores hemos llegado a un escenario donde un proxy inverso actúa de intermediario entre dos servidores web Nginx, balanceando la carga entre ellos.

Ya dijimos que una importante función que podía tener un proxy inverso era realizar el cifrado y descifrado de SSL para utilizar HTTPS en los servidores web. De esta forma se aliviaba la carga de trabajo de los servidores web, ya que es una tarea que consume recursos.

En definitiva, tendríamos un esquema como este:

Nginx SSL Termination



Podría llegarse a pensar que en términos de seguridad no es adecuado que el tráfico de red entre el balanceador de carga y los servidores web vaya sin cifrar (HTTP). Sin embargo, pensando en un caso real, la red privada y el proxy inverso/balanceador de carga, además de estar en la

misma red privada, suelen estar administrados por las mismas personas de la misma empresa, por lo que no supone un peligro real que ese tráfico vaya sin cifrar.

Podría cifrarse si fuera necesario, pero entonces pierde sentido que el proxy inverso se encargue del cifrado SSL para HTTPS, ya que haríamos el mismo trabajo dos veces.

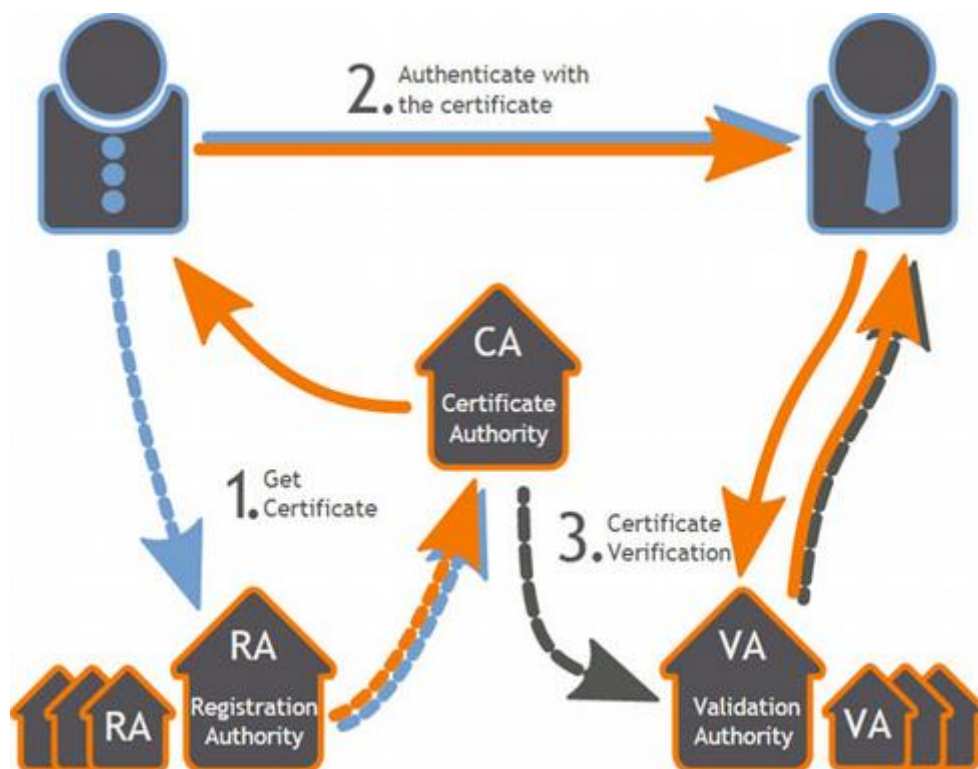
Así las cosas, nos quedaremos con el esquema de la imagen de más arriba para la práctica.

Certificados

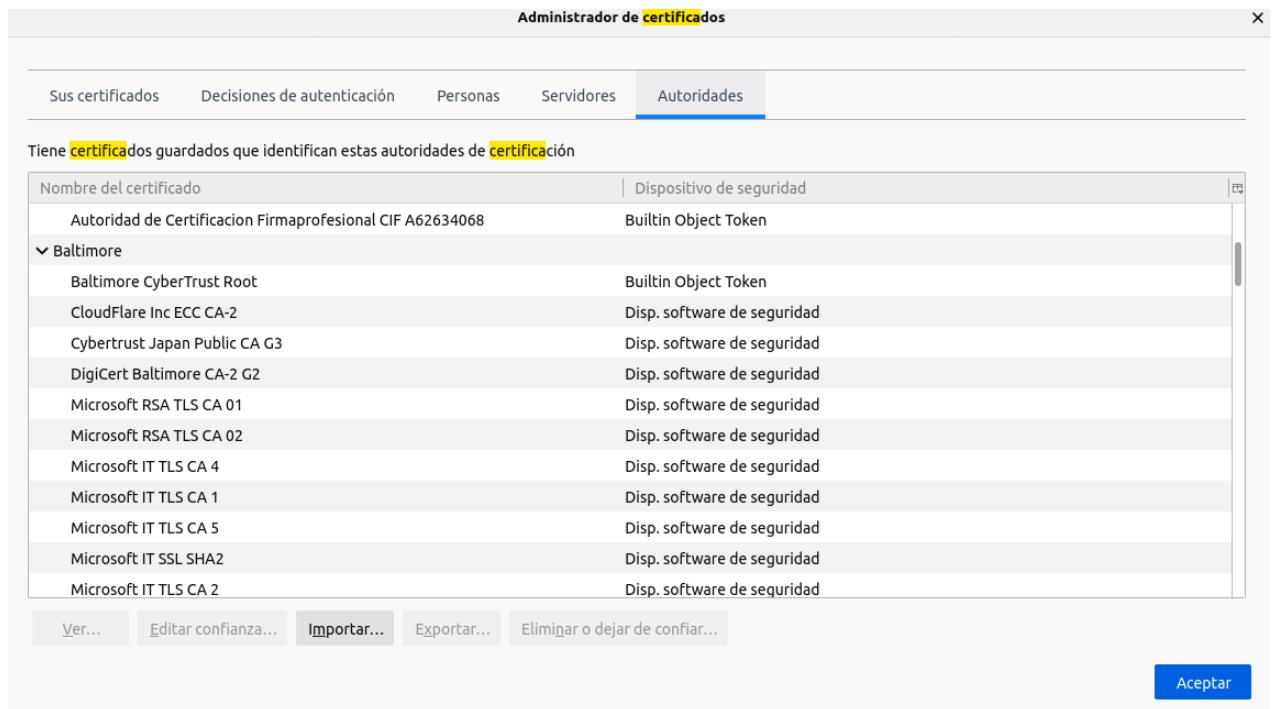
HTTPS se basa en el uso de certificados digitales.

Grosso modo, cuando entramos en una web vía HTTPS, ésta nos presenta un certificado digital para asegurar que es quién dice ser. ¿Cómo sabemos que ese certificado es válido? Debemos consultar a la Autoridad de Certificación (CA) que emitió ese certificado si es válido.

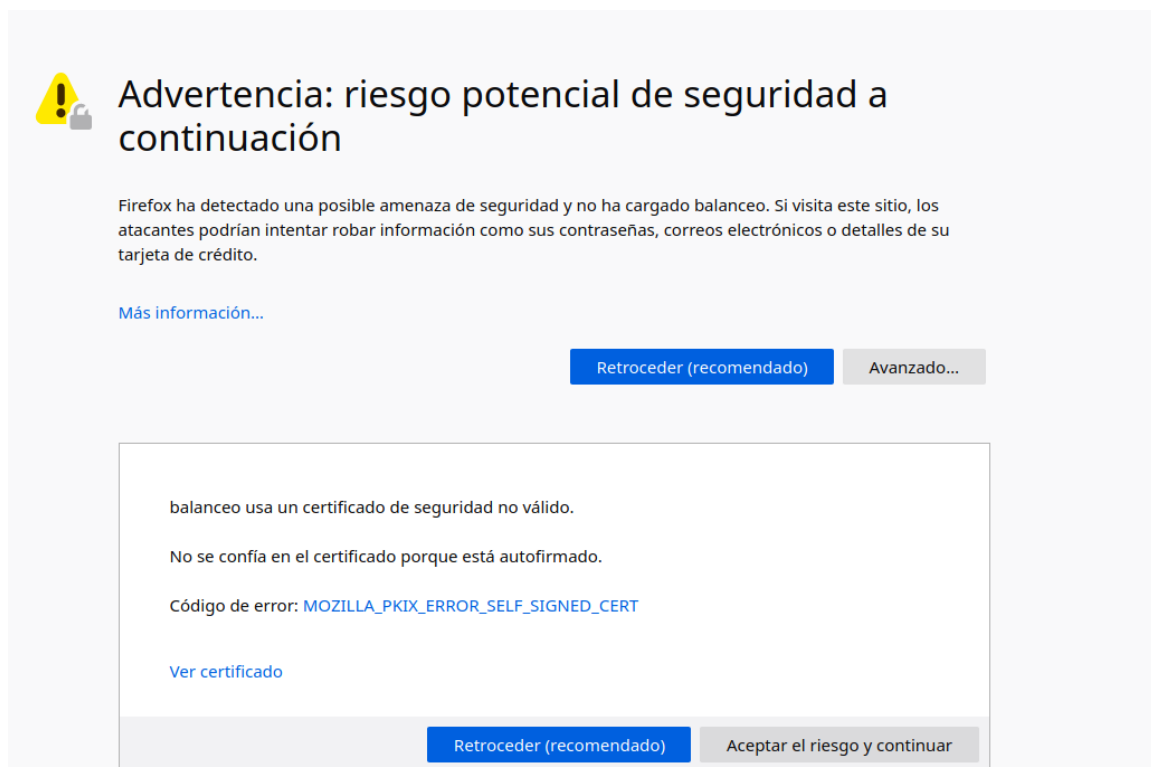
Las CA son entidades que emiten certificados y su funcionamiento se basa en la confianza. Confiamos en que los certificados emitidos y firmados por esas entidades son reales y funcionales.



Los navegadores web tienen precargadas las Autoridades de Certificación en las que confían por defecto a la hora de navegar por webs HTTPS:



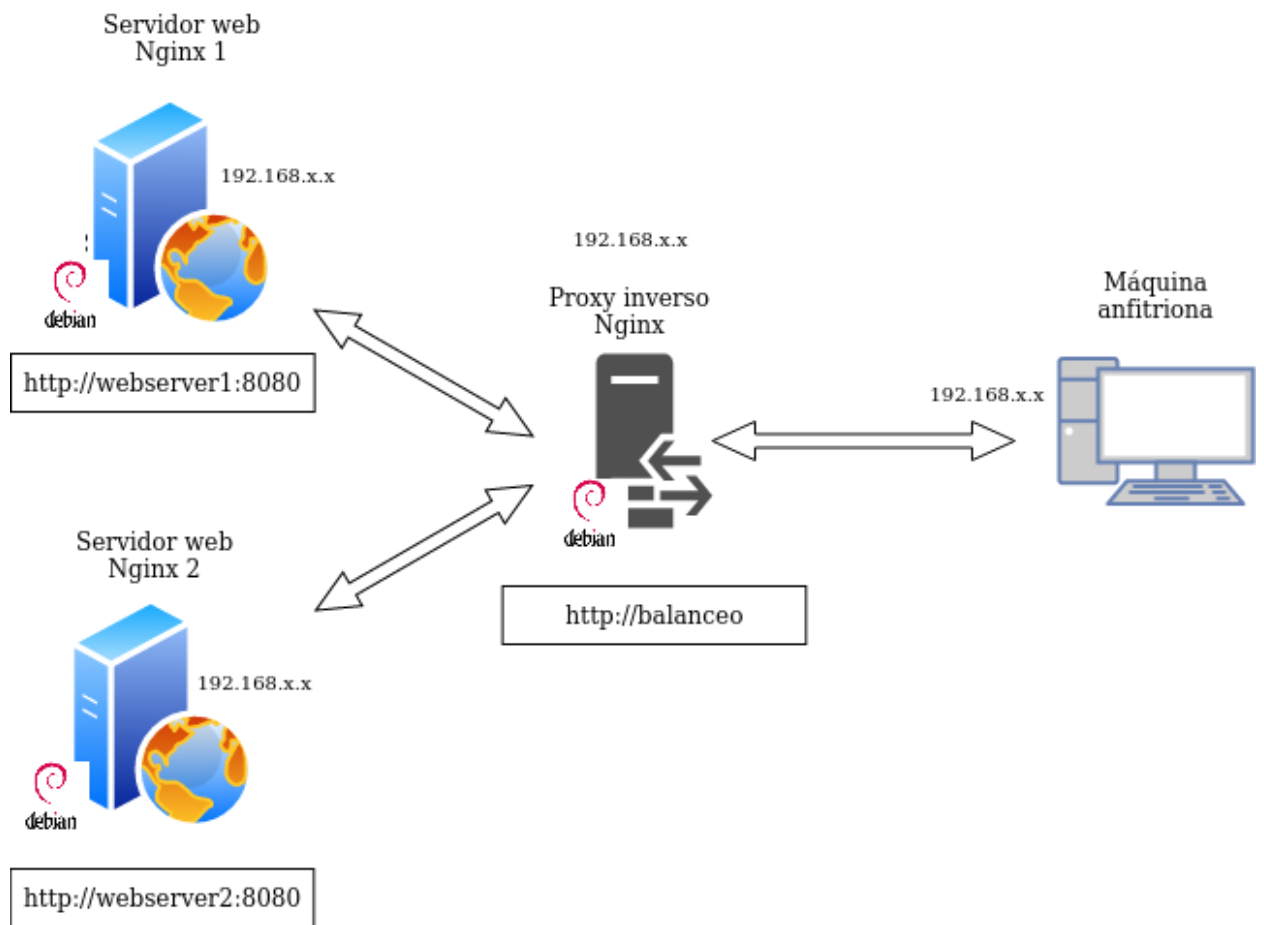
Si accedemos a una web cuyo certificado no haya sido emitido y firmado por una de estas entidades, nos saltará el famoso aviso:



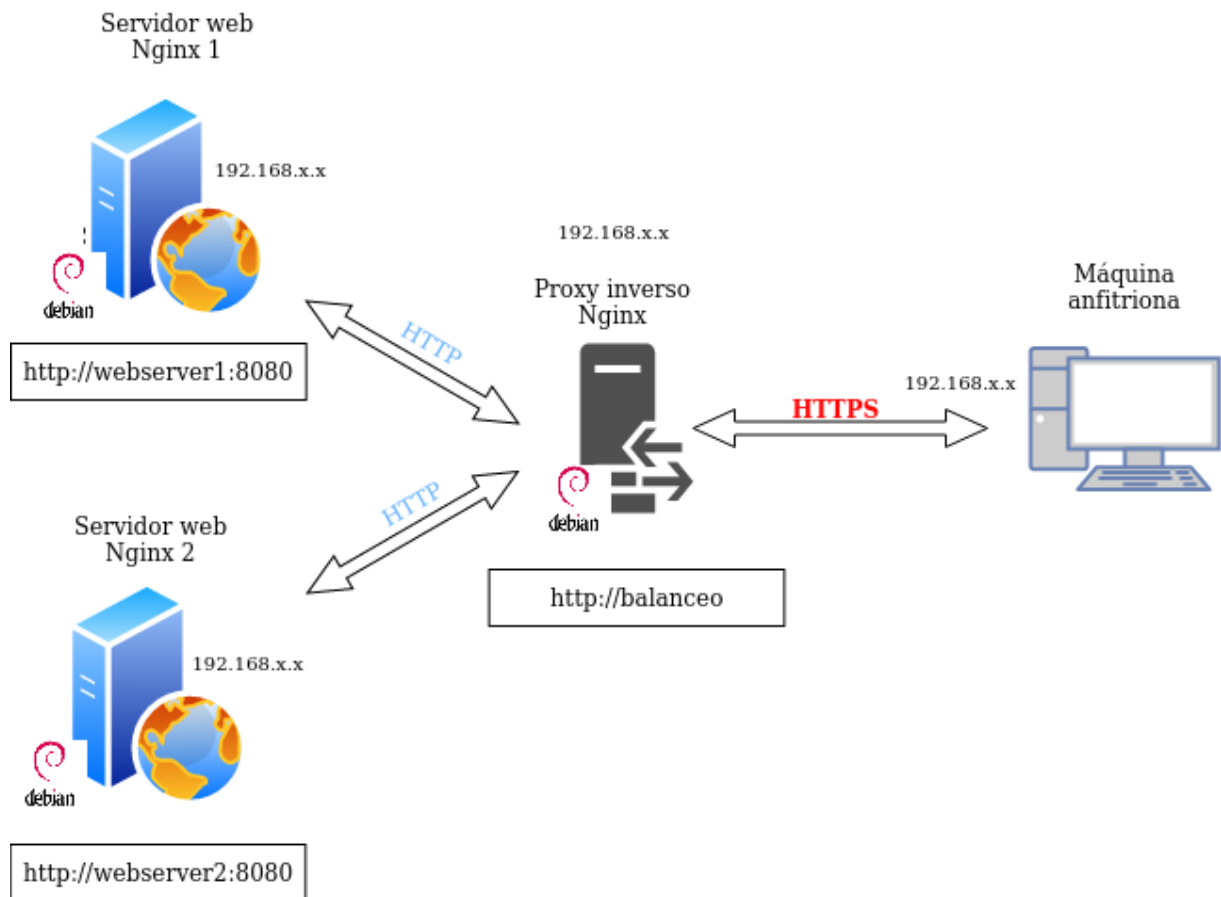
Ya que, si el certificado no ha sido emitido y firmado por una CA de confianza, puede que se trate de una web maliciosa que nos suponga un riesgo de seguridad, como bien dice el aviso.

Tarea

Partimos de la configuración exacta de la práctica anterior, que recordemos era esta:



Por lo que en esta práctica simplemente debemos añadir la configuración SSL para el cifrado en el Proxy Inverso:



Tal y como quedará la configuración, desde el cliente aún podríamos acceder a los dos servidores web con HTTP (podéis probarlo) pero es algo que solucionaremos en siguientes temas, configurando un firewall para que sólo la IP del proxy inverso pueda acceder por HTTP a los servidores web y nadie más.

Creación de certificado autofirmado

Nosotros no utilizaremos certificados de ninguna CA de confianza, básicamente porque:

- Nuestros servicios no están publicados en Internet
- Estos certificados son de pago

Así pues, nosotros crearemos nuestros propios certificados y los firmaremos nosotros mismos como si fuéramos una CA auténtica para poder simular este escenario.

Warning

Esto provocará que cuando accedamos por HTTPS a nuestro sitio web por primera vez, nos salté el aviso de seguridad que se comentaba en la introducción.

En este caso no habrá peligro puesto que estamos 100% seguros que ese certificado lo hemos emitido nosotros para esta práctica, no hay dudas.

Veamos pues el proceso para generar los certificados y las claves asociadas a ellos (privada/pública). En primer lugar debemos crear el siguiente directorio:

```
/etc/nginx/ssl
```

Podemos crear el certificado y las claves de forma simultánea con un único comando, donde:

- **openssl**: esta es la herramienta por línea de comandos básica para crear y administrar certificados, claves y otros archivos OpenSSL.
- **req**: este subcomando se utiliza para generar una solicitud de certificados y también solicitudes de firma de certificados (CSR).
- **-x509**: Esto modifica aún más el subcomando anterior al decirle a la herramienta que queremos crear un certificado autofirmado en lugar de generar una solicitud de firma de certificado, como sucedería normalmente.
- **-nodes**: Esto le dice a OpenSSL que omita la opción de asegurar nuestro certificado con contraseña. Necesitamos que Nginx pueda leer el archivo sin la intervención del usuario cuando se inicia el servidor. Una contraseña evitaría que esto sucediera ya que tendríamos que introducirla a mano después de cada reinicio.
- **-days 365**: esta opción establece el tiempo durante el cual el certificado se considerará válido. Lo configuramos para un año.
- **-newkey rsa: 2048** : Esto especifica que queremos generar un nuevo certificado y una nueva clave al mismo tiempo. No creamos la clave necesaria para firmar el certificado en un paso anterior, por lo que debemos crearla junto con el certificado. La **rsa:2048** parte le dice que cree una clave RSA de 2048 bits de longitud.

- **-keyout:** este parámetro le dice a OpenSSL dónde colocar el archivo de clave privada generado que estamos creando.
- **-out:** Esto le dice a OpenSSL dónde colocar el certificado que estamos creando.

El comando completo sería así:

```
raul@debian-server:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/nginx/ssl/server.key -out /etc/nginx/ssl/server.crt
Generating a RSA private key
.....+++++
.....+++++
writing new private key to '/etc/nginx/ssl/server.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Alicante
Locality Name (eg, city) []:Elche
Organization Name (eg, company) [Internet Widgits Pty Ltd]:IES Severo Ochoa
Organizational Unit Name (eg, section) []:2DAW - DEAW - RAUL
Common Name (e.g. server FQDN or YOUR name) []:balanceo
Email Address []:invent@superinvent.com
raul@debian-server:~$
```

Os solicitará que introduzcáis una serie de parámetros, como véis en el recuadro rojo de abajo de la imagen. Debéis introducir los mismos parámetros que en la imagen excepto en el “Organizational Unit Name” que véis recuadrado en amarillo. Ahí deberéis poner 2DAW – DEAW - Vuestronombre

Configuración SSL en el proxy inverso

De la práctica anterior, dentro del directorio `/etc/nginx/sites-available` ya debéis tener el archivo de configuración llamado “balanceo”. Es precisamente aquí donde realizaremos la configuración para que el acceso al sitio web se realice mediante SSL (HTTPS).

Dentro del bloque `server {...}` debéis cambiar el puerto de escucha (`listen 80`) por lo que véis en la imagen de abajo, añadiendo las siguientes líneas de configuración también, de tal forma que quede:

```
listen 443 ssl;
ssl_certificate /etc/nginx/ssl/server.crt;
ssl_certificate_key /etc/nginx/ssl/server.key;
ssl_protocols TLSv1.3;
ssl_ciphers ECDH+AESGCM:DH+AESGCM:ECDH+AES256:DH+AES256:ECDH+AES128:DH+AES128:ECDH+3DES:DH+3DES:RSA+AESGCM:RSA+AES:RSA+3DES:!aNULL:!MD5:!DSS;
server_name balanceo;
access_log /var/log/nginx/https_access.log;
```

Donde le estáis diciendo que:

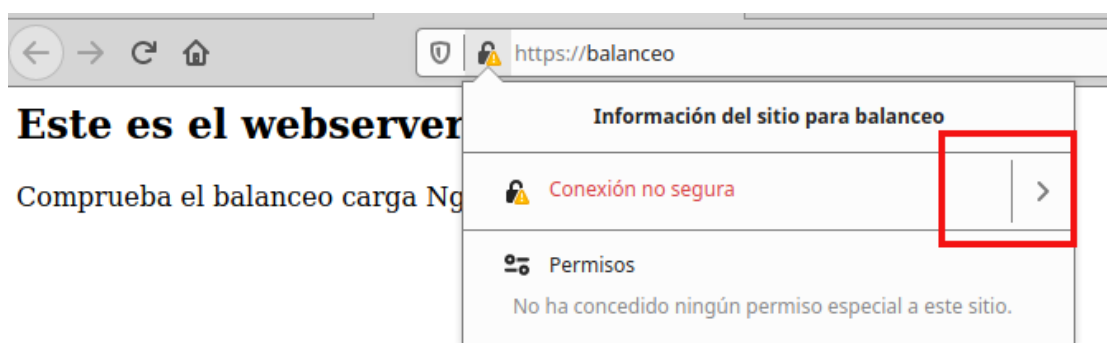
- Escuche en el puerto 443 → Puerto por defecto de HTTPS
- El directorio donde está el certificado que habéis generado anteriormente

- El directorio donde está la clave que habéis generado anteriormente
- Los protocolos y tipos de cifrados que se pueden utilizar → Estas son las versiones de protocolos y los tipos de cifrados considerados seguros a día de hoy (hay muchos más pero no se consideran seguros actualmente)
- `server_name` ya lo teníais de la práctica anterior, no hace falta tocarlo
- El archivo donde se guardan los logs cambia de nombre, ahora será `https_access.log`

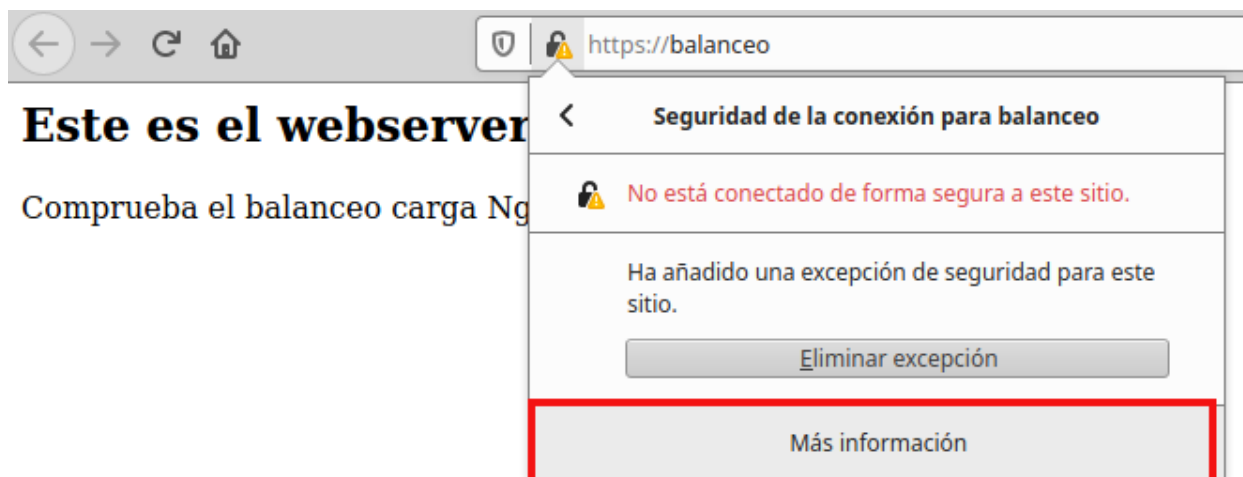
Recordad que tras modificar cualquier configuración de un servicio, hay que reiniciar el servicio, en este caso Nginx.

Comprobaciones

- Si accedéis ahora a `https://balanceo` os debería saltar un aviso de seguridad debido a que nuestro certificado es autofirmado, como comentábamos anteriormente.
- Si añadís una excepción podréis acceder al sitio web y recargando repetidamente la página con F5, veréis que el balanceo de carga se hace correctamente accediendo mediante HTTPS.
- Para comprobar que los datos del certificado son, efectivamente, los vuestros podéis comprobarlo así. Pulsando en el candado de la barra de búsqueda:



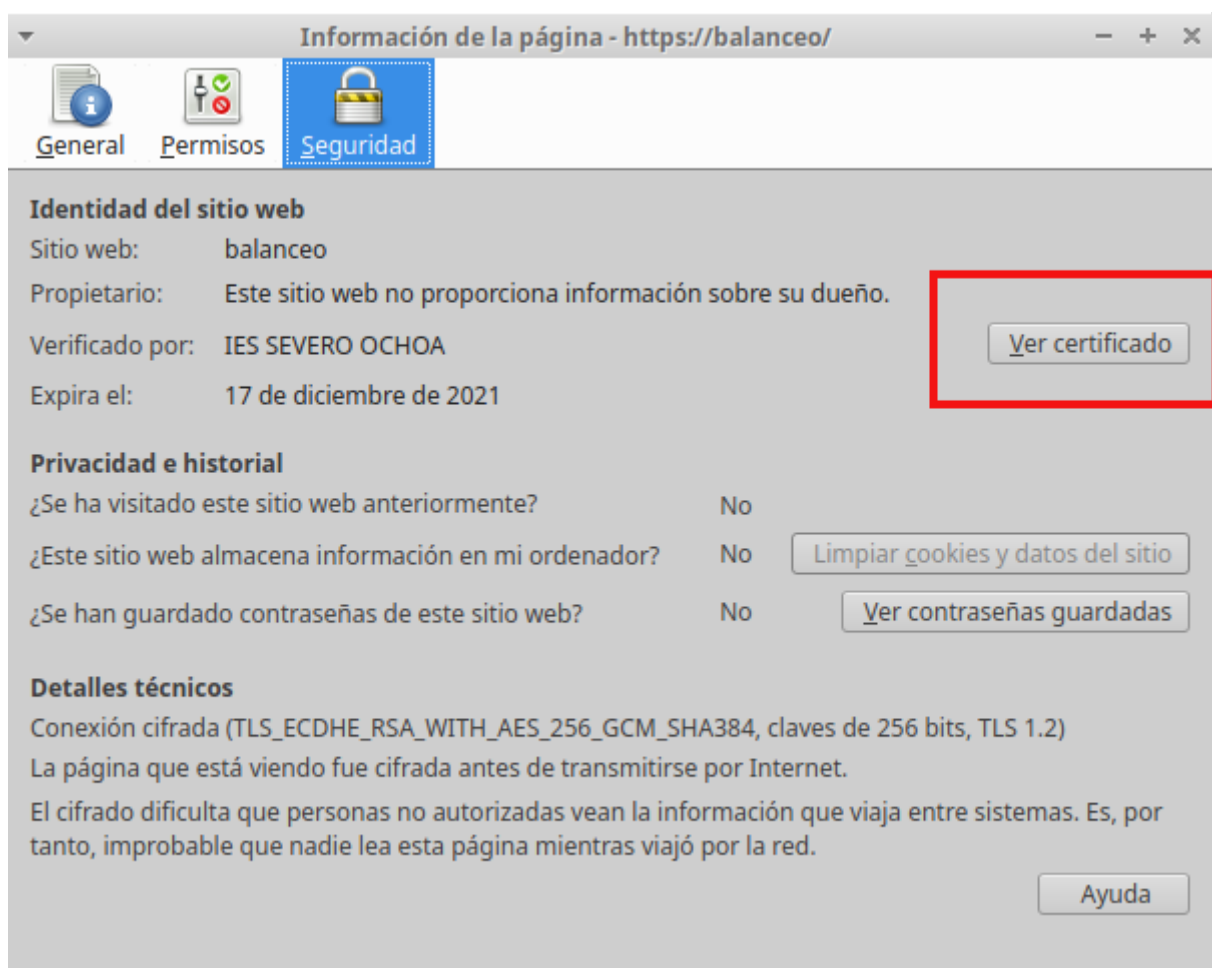
Con más información:



Info

Aquí también podréis eliminar la excepción que habéis añadido en la página de la advertencia de seguridad, por si necesitáis reiniciar las pruebas.

Y por último, ver certificado:



Y podremos ver los detalles:

balanceo	
Nombre del asunto	
País	ES
Estado/Provincia	Alicante
Localidad	Elche
Organización	IES Severo Ochoa
Unidad organizativa	2DAW - DEAW - RAUL
Nombre común	balanceo
Dirección de correo electrónico	invent@superinvent.com
Nombre del emisor	
País	ES
Estado/Provincia	Alicante
Localidad	Elche
Organización	IES Severo Ochoa
Unidad organizativa	2DAW - DEAW - RAUL
Nombre común	balanceo
Dirección de correo electrónico	invent@superinvent.com
Validez	
No antes	Sun, 16 Oct 2022 10:07:00 GMT
No después	Mon, 16 Oct 2023 10:07:00 GMT
Información de clave pública	
Algoritmo	RSA
Tamaño de la clave	2048
Exponente	65537
Módulo	B3:DC:25:05:2F:36:F9:9C:63:3A:A4:B0:93:52:10:83:83:F2:48:87:4C:E7:78:10:...

Si ahora intentáis acceder a <http://balanceo>, ¿deberíais poder acceder? Comprobadlo y describid qué pasa y por qué.

Redirección forzosa a HTTPS

Para que, indistintamente de la forma por la que accedamos al sitio web balanceo, siempre se fuerce a utilizar HTTPS, necesitaremos una configuración adicional.

Necesitamos añadir un bloque “server” adicional y separado del otro, al archivo de configuración de “balanceo”. Algo así:

```
server {
    listen 80;
    server_name balanceo;
    access_log /var/log/nginx/http_access.log;
    return 301 https://balanceo$request_uri;
}
```

Con esta configuración le estamos diciendo que:

- Escuche en el puerto 80 (HTTP)
- Que el nombre al que responderá el servidor/sitio web es balanceo
- Que guarde los logs de este bloque en ese directorio y con ese nombre
- Cuando se recibe una petición con las dos condiciones anteriores, se devuelve un código HTTP 301:
 - **HTTP 301 Moved Permanently** (Movido permanentemente en español) es un código de estado de HTTP que indica que el host ha sido capaz de comunicarse con el servidor pero que el recurso solicitado ha sido movido a otra dirección permanentemente. Es muy importante configurar las redirecciones 301 en los sitios web y para ello hay diferentes métodos y sintaxis para realizar la redirección 301.
 - La **redirección 301** es un código o comando insertado por un Webmaster que permite redirigir a los usuarios y buscadores de un sitio web de un sitio a otro.

Aclaración

Es decir, lo que estamos haciendo es que cuando se reciba una petición HTTP (puerto 80) en `http://balanceo`, se redirija a `https://balanceo` (HTTPS)

Tarea

- Eliminad del otro bloque `server{...}` la líneas que hagan referencia a escuchar en el puerto 80 (`listen 80...`).
- Reiniciad el servicio
- Comprobad ahora que cuando entráis en `http://balanceo`, automáticamente os redirige a la versión segura de la web.

- Comprobad que cuando realizáis una petición en el archivo de log `http_access.log` aparece la redirección 301 y que, de la misma manera, aparece una petición GET en `https_access.log`.

Cuestiones finales

Cuestión 1

Hemos configurado nuestro proxy inverso con todo lo que nos hace falta, pero no nos funciona y da un error del tipo `This site can't provide a secure connection, ERR_SSL_PROTOCOL_ERROR`.

Dentro de nuestro server block tenemos esto:

```
server {
    listen 443;
    ssl_certificate /etc/nginx/ssl/enrico-berlinguer/server.crt;
    ssl_certificate_key /etc/nginx/ssl/enrico-berlinguer/server.key;
    ssl_protocols TLSv1.3;
    ssl_ciphers
ECDH+AESGCM:DH+AESGCM:ECDH+AES256:DH+AES256:ECDH+AES128:DH+AES:ECDH+3DES:DH
+3DES:RSA+AESGCM:RSA+AES:RSA+3DES:!aNULL:!MD5:!DSS;
    server_name enrico-berlinguer;
    access_log /var/log/nginx/https_access.log;

    location / {
        proxy_pass http://red-party;
    }
}
```

Cuestión 2

Imaginad que intentamos acceder a nuestro sitio web HTTPS y nos encontramos con el siguiente error:



Your connection isn't private

Attackers might be trying to steal your information from **errorsmash.com** (for example, passwords, messages, or credit cards).

NET::ERR_CERT_REVOKED

Advanced

Refresh

Investigad qué está pasando y como se ha de solucionar.

Evaluación

Criterio

Creación correcta del certificado **1 pto**

Configuración SSL correcta del proxy **3 ptos**

Comprobaciones **2 ptos**

Configuración correcta de la redirección forzosa a HTTPS y comprobaciones **1 pto**

Cuestiones finales **2 ptos**

Se ha prestado especial atención al formato del documento, utilizando la plantilla actualizada y haciendo un correcto uso del lenguaje técnico **1pto**